

UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA
Denkleiers • Leading Minds • Dikgopolo tša Dihlalefi

COS700 Research Proposal

Gradient oscillations of Muon and Adam under optimal and suboptimal loss outcomes

Student number: u25735056

Supervisor(s):
Prof. Anna S. Bosman
Prof Katherine Malan

Table of contents

Abstract 2

Keywords: 2

1 Introduction 3

2 Problem Statement 4

3 Background 6

3.1 Optimiser Formulas . . . 7

3.1.1 Adam . . . 7

3.1.2 Muon . . . 10

3.2 Gradient oscillations and their implications . . . 13

3.2.1 Definition of the gradient, beyond derivatives . . . 13

3.2.2 How oscillatory gradients correlate to loss . . . 13

3.2.3 Gradient correlation oscillation tracking [36] . . . 16

3.2.4 Per weight oscillation oscillation tracking [31] . . . 22

3.3 Inductive learning bias types . . . 24

3.3.1 Simplicity bias . . . 25

3.3.2 Spurious feature bias . . . 26

3.3.3 Distributional bias . . . 26

3.3.4 Other relevant biases . . . 27

3.4 Chronology of studies: Muon and Adam under inductive biases . 27

3.4.1 The simplicity bias of Adam compared to SGD [46] . . . 28

3.4.2 Muon's Equally Learns Imbalanced Data Modalities [45] . 30

3.4.3 When Muon's Lack of Simplicity Bias Hurts its Performance [47] . . . 34

4 Methodology 37

4.1 Prerequisites . . . 37

4.1.1 Oscillations tracking methodology . . . 37

4.2 Experiments . . . 43

4.2.1 Subgroup robustness setting . . . 43

4.2.2 Class imbalanced CNN and RNN setting . . . 44

4.2.3 Spurious pixel CNN setting . . . 45

4.2.4 Class imbalanced transformer setting . . . 46

4.2.5 Appendix Reporting . . . 46

4.3 Libraries and frameworks . . . 46

5 Planning 47

6 References 48

Abstract

Adam has been a popular standard when gradient based optimisation is required in neural network parameter training. It uses an adaptive per parameter update size, in contrast to optimisers with less adaptation or a uniform update size for all parameters. A new optimiser, Muon, has outperformed or performed comparably to Adam in competitive benchmarks for small scale computer vision, and large language model tasks. As opposed to extensive per parameter adaptation, Muon proposes a larger view where each trainable parameter's update lies within its innate 2D matrix layer. These are then efficiently reshaped into a near orthogonalized distribution. The design intent is to address an empirical structural bias found in Adam of rank collapse. This is hypothesised to lead Adam to not learn rich, complex gradient distributions as maximally as possible. Related research has emerged evaluating the loss performance of Adam and Muon in ability to learn aspects of complex feature distributions, more rarely prioritised by non-adaptive optimisers. These optimality testing settings provide an opportunity to study the evolution of other patterns in training time. This includes how the trajectory of an optimiser's gradient updates in high dimensional space correlate to the loss trajectory. We explore one lens to study gradient updates as related to loss: gradient oscillations. This phenomena predicts divergence in convex function optimisation. In the non-convex landscapes of deep neural networks, counter intuitive phases have emerged where certain phases of oscillations can in fact correlate to the largest overall reductions in loss, or not predict complete divergence from the learning task. We therefore aim to comparatively study the evolution of first order gradient oscillations as they correlate to the loss trajectories of Adam and Muon on training tasks where they produce a specific set of richer learning abilities to non-adaptive optimisers.

Keywords:

Gradient oscillation, Adam optimiser, Muon optimiser, stability analysis, Feature learning bias

1 Introduction

For just over a decade the "Adaptive Moment Estimation" optimiser, or *Adam*, [1] has dominated neural network parameter training pipelines in industry and research. This includes a variety of task loads such as transformer based models, large language models, computer vision, and more [2, 3, 4, 5, 6, 7]. The success of Adam is attributed to its ability to adapt a learning update size per weight in the network over time, as compared to the uniform learning rates of foundational predecessor optimizers such as stochastic gradient descent (SGD) [2, 4, 6]. Related first order derivative optimizers exist that apply their own versions of per-parameter adaptive estimation that have achieved similar competitive benchmarking results as Adam [4, 7].

A new optimiser has recently emerged that outperforms Adam in small scale CNN settings, and consistently in small to increasingly large LLM benchmarks [7, 8, 9, 10]: Muon [8, 11, 12]. Muon does so with only heavy ball momentum adaptive scaling per parameter [9, 11, 12, 13]. It then approximately orthogonalizes [14, 15, 16] the first order gradient (or derivative) updates of weights when arranged in a dense 2D matrix structure.

A primary goal in neural networks is minimisation of the chosen error or loss function on a learning task [2, 4, 6]. One technique for neural networks to fundamentally learn this is by being equipped with methods to change internal parameters known as "weights" [4, 6]. Due to the high dimensionality of weight inputs and problem complexity, optimizers face specific challenges of navigating the high dimensional geometry of a non-convex, non-smooth error or loss landscape [2, 4, 6, 17, 18, 19, 20]. First order derivative methods under random initialisation of parameters have achieved convergence successes with these optimisation barriers present [2, 4, 21]. The required conditions guaranteeing convergence to a certain level of optimality and training time frame are not yet fully understood but remain under broad study [17, 18, 21, 22, 23, 24].

Studies of optimal and sub-optimal convergence behaviours of first order methods exist due to these challenges. These sub-fields include loss landscape analysis, and analysis of optimiser algorithm dynamics, beyond predictions from known theories of optimisation [25, 26, 27].

In particular, the oscillatory dynamics of gradients have been a growing topic [27, 28, 29, 30, 31, 32]. It has been identified that periods of high loss oscillation in the short term can occur while in the long term loss is minimised the most as compared to a smooth loss decrease. This is termed "edge of stability training" (EOS) for non-adaptive optimizers. For adaptive optimizers such as Adam

a related phenomena was termed "Adaptive EOS" (AEOS) [30]. It has been proposed that certain patterns of *dynamical instability or stability* [33, 34, 35] may lead to optimal outcomes [34], with emerging studies taking different approaches of analysing patterns related to this EOS [31, 34, 36, 37, 38].

The (A)EOS period correlates with greater gradient oscillations. This is specifically inclusive of gradient greater step sizes measured that exceed theoretical thresholds that lead to divergence in convex optimisation settings [28, 29, 30, 32], based on the steepness of the curvature being measured for the point in the landscape where a step is taken. Adam shows evidence of being more able to frequently enter and converge at increasingly higher curvature regions than non-adaptive methods [30]. It is hypothesised but not concluded that this implicit instability generation and resolution scheme by Adam may be linked to poorer generalisation optimality compared to well tuned SGD when it better resolved these instabilities [24, 30, 39]. The behaviour of Muon in this context is not yet well investigated nor established.

Following from the literature, we aim to study behaviours related to dynamical stability contrasting Adam and Muon. This adds to the body of work investigating the dynamics of optimiser algorithms as related to optimal and non-optimal loss trajectory performance.

2 Problem Statement

First order gradient optimizers estimate how to use and extend upon gradient measures to minimise loss by choosing unique directions to move the parameters in to attempt arriving at an optimal loss outcome, however sub-optimal outcomes may result. Comparatively, studied experimental design findings do exist testing Adam and Muon at optimal and sub-optimal validation performance in a subset of feature learning bias related [40, 41, 42, 43, 44] experiments on specific datasets [45, 46, 47]. The focus of such studies is often on comparing Muon and Adam to SGD independently. Comparison of Muon and Adam in loss trajectories is brief including lack of rigorous motivation for hyperparameter tuning. Furthermore studies of the feature learning benefits of these optimisers focus on final measures of training and validation loss measures decoupled from studies of how gradient oscillatory regimes correlate to loss outcomes of individual optimisers.

Therefore we propose the following research questions that can be explored at feasible scale, to develop experimental constraints to be reported for our study:

1. **Oscillation tracking method choices:** What is the set of most computationally efficient methods to sample oscillations of high dimensional weight

space, consolidate results, and relate them to loss trajectory behaviour?

    1. How does a differing level of consolidated compared to granular methods of gradient oscillations assessment reveal small or big behaviours in scope? Examples of granularity may include: (a) Individual gradient sampling compared to consolidating the gradients in statistical or mathematical measures. (b) Train time sampling scope at each iteration time point or smoothed averages across time intervals. (c) Layer sampling across input, hidden, output layers or a global consolidated measure across all layers.
    2. What outcome does each level of granular sampling provide, and what is the cost to gain the insight, in terms of computational efficiency?
    3. For which train time phases do the proposed measurement techniques consistently explain the loss trajectories unique to the performance of the optimisers?

2. **Optimiser hyperparameter tuning choices:** What are the optimal hyperparameter tuning regimes to use on the optimizers for each task, to ensure balanced comparison of optimally tuned Adam and Muon for the same learning task?
    1. Which optimiser require more extensive tuning for each learning task to find optimality? Conversely, which optimiser requires less extensive tuning for the learning task?
    2. In what kind of tuning regime are our findings applicable, if our tuning is limited?
    3. Can desired learning behaviours be induced in the slice of hyper parameters tested for each optimiser to learning task combination?
3. **Dataset distribution choices:** What dataset distribution choices lead to optimal and comparatively sub-optimal learning performance between Adam and Muon?
    1. Do these learning tasks correspond to real datasets and challenging problems in machine learning to match real practitioner concerns?
    2. Beyond train loss on the overall dataset, which measures of error on tested feature attributes support hypotheses of gradient signals the optimiser was biased to exploit, such as per class error, or errors on specific validation dataset distributions?
4. **Architecture choices:** What neural network architecture choices lead to optimal and comparatively sub-optimal learning performance between Adam and Muon?

We may now finalise our main research questions after answering the prior building block “sub-questions”.

5. Given the chosen **combination of dataset, optimiser, architecture, and gradient oscillation measure:** How do oscillatory gradient behaviours of optimally tuned Adam and Muon correlate to their loss trajectory over train time?
    1. When are the gradient trajectories of Adam and Muon significantly different or similar, and do they result in similar or different loss trajectories?
    2. To what extent to gradient trajectories infer the priorities of direction to step in based on feature attributes optimisers to minimise loss?
    3. In general, do phases of phases of greater or reduced gradient oscillations correlate to greater or reduced overall loss respectively, within a train time period?
    4. If the prior hypothesis does not hold, do gradient oscillations result non-trivial, non-intuitive loss trajectories, where consistent or inconsistent? What specific types of patterns result to characterise loss behaviours?
    5. When the complexity or the rarity of the feature learned is known, do oscillatory gradient dynamics further give insight into the different gradient directions chosen per train time step and how quickly and smoothly overall loss is minimised?
    6. How does train loss and oscillatory trajectories correspond with validation loss, which provides an insight into real world performance? Do the train loss and oscillation correlate to indicators of robust generalisation performance?

3 Background

The structure of discussion is as follows:

1. In Optimiser Formulas we discuss the proposed update formulas on weights by Adam and Muon. We briefly discuss studies to explain each major aspect of their problem solving abilities in areas with popularity of their usage.
2. In Inductive learning bias types we discuss dataset and/or architecture design settings that report sub-optimal or optimal loss performance of Adam, Muon or both, beyond theoretical expectations.
3. In Gradient oscillations and their implications we discuss oscillatory gradient studies in connection to learning performance as measured by loss and extended accuracy measures. We focus particularly on methods that apply to studying Adam or Muon at the scale of architecture and dataset

settings we desire.

3.1 Optimiser Formulas

One technique for neural networks to fundamentally learn is by being allowed to change internal numeric parameters known as "weights" after a human practitioner has set constraints on the rest of the helper components of the learning architecture [4, 6]. A range of optimisers have been devised in the field of deep neural networks to equip it the ability to find a desired set of weights, typically such that they have the efficient ability to learn both training data and generalise to unseen data [2, 6]. Present throughout navigation of high dimensional weights over the error surface generated by different configurations of them, are multiple local minima traps that require escaping from, in favour of more optimal minima even if a global minimum cannot be found [2, 4, 6]. Locations may be settled to that optimise loss even if not minima termed saddle points [19, 20]. Gradient based optimisers use derivative signals that quantify how loss changes as a weight is shifted by an extremely small positive, while other weights that would cause varying outputs are kept frozen [2, 4]. This is specifically termed the *partial* derivative. Using estimations from these derivative measures optimisers have found success in optimising over different problem domains. This stems from empirical, heuristic based tools and connections to understood theory as connected to ideas from different fields of convex and non-convex optimisation from high input dimensions.

3.1.1 Adam

For each scalar weight or any other trainable parameter in the network, Adam intended to update each weight by a custom estimate unique to each weights traversal in the high dimensional loss optimisation landscape. This is as opposed to the uniform learning rate size of the classical stochastic gradient optimiser in neural networks [1]. Adam is part of a family of adaptive optimisers which create different measures purely from the same first derivatives of each of the weights to to perform the update [2, 4, 6, 7]. These derivative are often "imagined" as arranged in a list like structure known as a gradient vector, used in the fields of linear algebra and computer science [4]. We can then imagine a gradient vector performing an update on the weights in sync at a single training iteration time step based on the update stored for each weight. Adam is directly inspired by predecessor methods, known as AdaGrad, and RMSProp [48, 49].

AdaGrad showed success in sparse gradient optimisation. In other words, when gradient learning signals are rare from certain features, AdaGrad could boost learning of rare signals which may be beneficial for some learning settings. A

side effect however is that due to its mathematical construction, large gradient signals eventually become dampened until reduced to effectively 0 and learning stopping as training continues.

RMSProp showed success in on-line and non-stationary settings, by modifying from AdaGrad. Instead of keeping a long or full history of previous (squared) gradients accumulated. It instead uses a moving average such as the exponentially decaying average. Learning is barely dampened to the extent of AdaGrad, and the decaying average means RMSProp maintains the most relevant past history of (squared) gradient updates during continuous training. This means in on-line settings, where new batches of data are received for training and therefore may define that the training objective has shifted, RMSProp can adapt to this scenario, therefore the utility of the history of gradient accumulations is not damaging for continuous training. However it does not boost sparse gradients as effectively as RMSProp.

Adam aims to combine the benefits of both algorithms. It stores a moving average of past gradients in its momentum term. This preserves the magnitude and direction of gradients. The momentum term can be intuitively thought of as a ball in physics moving with momentum. If the term gets receives a gradient signal to move in a direction, and inspects the accumulation of gradient signals to find they same direction was pointed in at a magnitude, Adam will accelerate the descent of the weight based off the history showing the direction consistently optimises loss. If the inspected history shows an opposite direction taken, Adam will decelerate descent by taking some influence of the opposite past gradient. Adam's formulation of momentum and variance decides what "weight", or level of importance, to give the past historical gradient measure and the present update to take. This intuitively decides the compromise over how much of the current direction to value or should overpower the past history accumulated.

Adam then stores a moving average of past squared gradients in its variance term and of gradient accumulations in the momentum term. The squared gradient ignores the direction of the gradient update, and captures an idea of the absolute distance magnitude from a 0 centre. More rare gradient signals receive larger updates than larger gradient signals, however the progression of dampening learning to zero does not result due to a moving average being used.

The use of a moving average, where Adam uses exponentially decaying average up to the beginning of training, means Adam can therefore adapt to online and non-stationary settings, as well boost rare directions with the momentum term, if the same gradient directions are consistently optimised over. Authors further claim this setting helps promote faster convergence if the same gradient

directions are consistently chosen and reduce oscillations if an opposite present direction is used compared to past updates, as claimed for Adam, a phenomena termed step size annealing [1].

Adam uses its decay rate β_1, β_2 and a counterpart $1-\beta_1, 1-\beta_2$ to control these weighted values on the past and present history. This description of these rates as weights is not to be confused with neural network weight parameters which also dictate an idea of weighted value but of input data features.

The final key proposition of Adam is the bias correction forms of the momentum and variance terms. This corrects for a bias of exponential decay where at the start of learning, there exists no history of past gradients, and gradient signals at a present moment may be ignored and update size is extremely small. Bias correction means that despite a lack of past history, the momentum and variance terms are appropriately scaled to not be biased towards 0, and instead account for the importance of the present value. This bias correction scales appropriately when either the past or present gradient accumulation should be recognised more or less. For example, a minimal boost is given to the momentum if the weight of the past gradient is low and the greater weight of the present gradient was to be used.

Adam classically uses a formulation of the (bias corrected) momentum, variance terms with a uniform learning rate set per parameter, and the adaptation comes from the history of the gradient estimation measures that act with the learning rate.

Adam updates weights with the following formula [1]:

$$
\begin{aligned}
g_t &= \frac{\partial l(w_t)}{\partial w_t} \\
m_t &= (1-\beta_1)g_t + \beta_1 m_{t-1} \\
v_t &= (1-\beta_2)g_t^2 + \beta_2 v_{t-1} \\
\hat{m}_t &= \frac{m_t}{(1-\beta_1^t)} \\
\hat{v}_t &= \frac{v_t}{(1-\beta_2^t)} \\
w_t &= w_{t-1} - \left(\eta * \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} \right)
\end{aligned}
$$

Here t and $t-1$ is the current computation time step and the previous, respectively, w is the numeric weight value, g is partial derivative of weight w with respect to that of some loss function l, η represents the "learning rate" or "step size", m is momentum , v_t is variance (both sometimes referred to as the "first moment"

and "second moment", respectively), and finally we have that β_1, β_2 are the exponential decay rates. $\hat{m}$, $\hat{v}$ are the bias corrected versions, of momentum and variance respectively. ϵ is a numerical stability factor to prevent division by values very close to 0 that a computer may round to 0.

An understanding of the interaction of the full scope of all these heuristically derived, problem dependent [1, 50] parameters of Adam is neither trivial, nor fully understood, although benchmark studies have emerged probing this using extensive hyperparameter sweeps over *all* of Adam's parameters and characterising certain dynamics [4, 7, 50]. Due to this intensive tuning regime, historically, studies may freeze certain hyper parameters of Adam based off defaults or heuristics to study baseline Adam behaviour in comparison to new optimizers. And this heuristic driven practice forms the very basis of criticism that new optimizers must be proposed under the equivalent of competitive tuning unless only a slice of Adam's behaviour is being studied [4, 7, 8, 50]. Let us take for example an often overlooked parameter of Adam for example, ϵ . In the literature benchmarking Adam performance this hyperparameter has historically been set to default values in the literature, or its explicit value is omitted, and rarely tuned over. Under hyperparameter tuning of the value, performance gains can be yielded, implying the possibility of overlooked properties and problem dependent benefits beyond the oversimplified currently given even in official popular machine learning library documentations such as Tensorflow [4]. Adam's full range of adaptive dynamics to estimate problem solutions therefore continue to be under formal investigation.

3.1.2 Muon

Muon is a first order optimiser like Adam as it uses first order gradient measures to perform its update. Specifically, it computes a weighted average of past gradients, similar to Adam. Muon however may use one traditional version called heavy ball momentum [51, 52], that does not discount the current gradient update, and instead uses the full gradient update combined with a discounted past accumulation. As we extend, Muon's official implementation [9] uses a form of momentum termed Nesterov, or "lookahead" momentum [52]. Intuitively, lookahead momentum involves an optimiser taking the gradient not at the current weight value, but at an estimated future position relative to the current weight, and adjusting its current update based on the future expectation. This setting of momentum is claimed to perform empirically best with Muon although the dynamics are not yet well explained theoretically for Muon specifically [8, 11].

The proposed unique design intention of Muon is to alter certain distributions of Adam's weight update matrices due to a bias observed in them, similarly with

other element wise gradient optimisers. This is a reshaping of the baseline first order gradient distribution. Optimisers such as Adam and SGD-momentum are found to have what is called a high condition number. In linear algebra this means the 2D weight matrices for Adam, are composed of basis vectors that are linearly dependent (low rank), and highly imbalanced in basis vector size [8, 10, 11, 12].

To understand what basis vectors are we must first briefly understand that a matrix in linear algebra is a transformation that takes an input vector of a certain dimensionality, to an output vector of a certain dimensionality [53]. A matrix of dimensionality $n \times m$, of row and column dimensions respectively, will denote that it is composed of m basis vectors each of dimensionality n. A matrix transform, or a matrix multiplication, is then a linear combination (i.e. a weighted sum) of these basis vectors. The "weights" are each consecutive value component of the input vector. Each component in the input vector is therefore the scalar weight of the respective basis vector in the ordered sequence of column vectors in the matrix. At a high level the takeaway is that a matrix is not simply a grid of numbers, but a set of basis vectors, that accept input, and send the input to an output vector after being linearly combined. The output vector matches the dimensionality of each basis vector due to the weighted addition operation.

To understand linear independence we have that certain basis vectors can be a linear combination of other basis vectors. At a high level this means that the basis vector encodes redundant information. The same output vector the linearly dependent basis vector may send input vectors to can be encoded instead by existing linearly independent basis vectors. This results in efficiencies, when learning could be spread independently between the basis vectors. The weight basis vectors of a 2D weight matrix are tied to the features and therefore control dependent or independent directions of learning of features.

A matrix can be decomposed into a series of "simpler" transformations, it can be discovered what are termed the singular values of the matrix. These represent the strength of a stretch factor in independent directions that the matrix applies to the input vector components when they pass through the series of decomposed transformations, as they reach the output vector dimensionality. It is here that in singular value analyses that an imbalance of direction strengths can be discovered, and an exact count of how many directions in the original matrix are linearly dependent to all other basis vectors.

What Muon does is perform an efficient method to find the *approximate* nearest orthogonalised version of the original *gradient update* matrix that is then applied

to the weight matrix. In a decomposition method such as singular value decomposition, the singular values a middle decomposition matrix can be revealed and set to 1 (or approximately so), while the remaining left and right matrices are orthogonal. When the decomposed matrices are combined again to form an equivalent transformation matrix, the result is a matrix whose basis vectors are independent (non-redundant), and unit length i.e. length 1. The matrix approximates the transformations of the original matrix while having balanced and independent distribution of gradient signals. Muon's authors formally state that perfect orthogonalisation with their proposed Newton-Schultz iterative method [15, 16] is not required to recover the benefits of Muon, of balancing direction updates, and indeed other studies have found this [10, 11, 12]. This saves computational efficiency for the Newton-Schultz algorithm.

It is not well understood why orthogonalisation is beneficial at the first order gradient level. One hypothesis is that when Adam fails to identify inefficient learning redundancies, and rare directions at the level of the gradient distributions of the matrix Muon recovers these gaps [8, 10, 11, 45, 54]. Adam only has the view over individual elements to approximate boosting of rare directions and dampening of strong signals, if learning rare directions is indeed beneficial, over preferring strong obvious signals in data to form decisions. Some related results of directional learning are found in empirical studies known as spectral analysis, which is when the matrix transformations are decomposed into smaller, simpler pieces and their components are analysed. In such studies of Muon the singular values from SVD decomposition can be analysed [8, 10, 11, 12], although such a decomposition remains expensive, as Muon's orthogonalisation scheme sends singular values to 1 without ever knowing what they are [11]. This is therefore an efficient but opaque method. In such spectral analyses it is found that Muon has a more diverse distribution of singular values of the weight matrices, meaning the benefits are found to transfer over from orthogonal gradients to form more diverse learning representations. Muon is stated to be related to a series of optimisers that are termed "spectrally aware" [7, 8, 14, 55], and may either operate on the first or second order gradients, however such methods have not reached Muon's level of success owing mainly to the computational expense of operating with spectral components of matrices.

Muon updates weights with the following main formula using standard heavy

ball momentum as from [8, 11, 12]:

$$
\begin{aligned}
G_t &= \nabla_w L_t(W_{t-1}) \\
M_t &= G_t + \mu M_{t-1} \\
O_t &= \text{Newton-Schulz5}(M_t) \\
W_t &= W_{t-1} - \eta_t O_t
\end{aligned}
$$

The subscripts $t, t-1$ are time steps of current and previous respectively, the layer matrix of weights W holds the individual elements w, G stores the first partial derivative of each corresponding weight element, ∇ is the partial derivative operator that acts on the loss function L with respect to each individual weight element w, L is the loss matrix function that computes output loss values, μ is the momentum decay rate, M is the matrix of momentum values, O is the orthogonalised momentum values with the Newton-Schultz5 iterations, η is the uniform learning rate over all parameters.

3.2 Gradient oscillations and their implications

We evaluate key studies evaluating the importance of gradient oscillations as they correlate to loss oscillations.

3.2.1 Definition of the gradient, beyond derivatives

Before this we first require briefly extending first order derivatives to the gradient vector. At a time iteration step t during training, the weights of the network can be intuitively imagined as a flattened vector list structure. The derivatives for each weight can be calculated and arranged in a corresponding vector. This is termed the gradient vector. This vector is used by these first order gradient optimisers to approximate the direction of steepest descent. Therefore to communicate the synchronous update of weights at a time step, the indicate θ as the weight vector, and ∇ as the gradient operator, L as the loss scalar given weight input of the network. Finally $\nabla L(\theta_t)$ denotes the gradient operator obtaining the partial derivatives of all weights with respect to loss. This represents the gradient vector. More precisely, this list structure can correspond with an interpretation of a formal vector from the field of linear algebra and other fields of optimisation theory. This induces useful mathematical properties.

3.2.2 How oscillatory gradients correlate to loss

Various studies have evaluated how oscillatory gradients link to optimisation performance measures at different levels of mathematical or experimental length and computational efficiency of methods.

A body of neural network edge of stability [27, 28, 29, 32, 36, 56, 57], and dynamical instability [27, 34, 35, 58, 59] literature exist finding that at training time for SGD and Adam there exist periods of greater oscillatory loss behaviours in the short term where overall decrease of loss in the long term still occurs [28, 29, 30, 32, 60]. More explosive gradient measures with greater gradient oscillations or spikes are recorded in this period for Adam, coupled with the finding that Adam can more frequently enter sharper curvature in the loss landscape than SGD [24, 28, 29, 30]. Adam when tuned to produce this behaviour is hypothesised to impact its performance ability, when well tuned SGD is found to generalise better for reasons still not well understood theoretically [30]. An important caveat is that studies have emerged demonstrating well tuned Adam can perform better than SGD on validation loss [4, 5, 7, 50]. Adam requires more rigorous hyperparameter tuning efforts which may be overlooked in industry, or research baselines [4, 7, 8, 50]. However, in these cases where Adam does surpass SGD, it has not been studied if a different oscillatory regime, perhaps one encountering greater sharpness than SGD, is a factor. At the second order gradient level of landscape curvature, studies have emerged analysing if this sharpness meaningfully correlates to specific loss outcomes in generalizability [61]. The study of [61] finds it does not have strong correlations in general except multiple cases where *sharper* minima obtain improved generalisation scores. This suggests whether oscillations and sharpness is measure of generalisation ability depends on the basis of a particular case where Adam may find optimal performance under tuning as studied by [54, 62]. In the study of [60], it is found that larger oscillatory regimes induced by large learning rates enable improved generalisation for learning weak and strong feature distributions, while smaller learning rates do not generalise well for either feature trait.

Other studies such as that of [26, 27, 28, 29, 30], evaluate the gradient oscillations as they correlate to measured encounters of sharper loss curvature, as further correlated to divergence of loss. The study of [26] notably formulate a differential equation of time with respect to weight that can form smoothed time series predictions of weight vector iterates, called central flows, and open source the tools to conduct the analyses. These central flows are a simulation, and can predict loss trajectory, gradient norms and other measures in the study that are a function of the weights, near the real training smoothed trajectories at that time point. The study exploits the evidence that weights during periods of oscillation may be highly correlated with divergence thresholds of sharpness detected that violate convex optimisation convergence. Weights can be imagined in a 2D setting where they can move positive or negative in value, until they settle at the optimal value in a snapshot of time. Over the local curvature of

weights may evolve over the third dimension of time, drawing a 3D like curvature. The study again finds that the periods of oscillation are strongly correlated with high sharpness measures detected in the direction of the weight by the Hessian. During this regime, it can be visualised that certain weights are stuck in convex or convex-like valleys, that evolve over time to remain narrower or wider valleys that still cause divergence. Hence we can imagine this drawing a 3D valley structure over the time evolution of a weight, and obtaining time averaged, or smoothed trajectories of weights instead of their exact oscillatory values.

The authors propose this lifts discussion beyond predicting erratic weight and loss behaviours due to chaotic oscillation schemes especially under convex settings especially under multiple eigenvalues violating the stability threshold as in the study of [63] instead of studies opting for assumption of one eigenvalues for simplicity.

Crucially the above formulations can enter periods breaking down, as recognised by the authors. Indeed it assumes their formulation from Taylor expansions applies for weights evolving in locally convex or near convex settings at a time snapshot, whereas weights may sometimes exist in a non-convex minimisation setting. Therefore periods of time where the sharpness values may be encountered that predict convex problem divergence, and not non-convex divergence. The authors state larger learning rates cause a break down in the predictive central flow. The findings apply only to the square error loss formula for ease of derivation reasons.

While theoretically rich, such forms of analysis are computationally expensive to observe to calculate the curvature with second order gradient Hessian matrices for high dimensional weights, and typically limit their study to smaller budgets in terms of architecture size, epoch run time, and non-linear functions used. The study of [26] even goes on to measure third order Hessians, motivating that expansion results in more robust formulation of their sharpness penalty term that uses oscillations and plays a role in predicting a weight at a time step, in addition to the first order gradient. This is how they modify the traditional Taylor expansion's curvature penalty term. This is acknowledged as a highly expensive operation however.

For Muon formally published studies do not yet exist to our knowledge, specifically demonstrating how oscillatory behaviours correlate to explain empirically optimal and non-optimal loss outcomes. Suboptimal outcomes exist that are not explained by formal recommendations of when to not use Muon [8, 50]. Formally use is not intended on on matrices that are non-dense such as the embedding matrix of transformers, nor on non-2D parameters due to Muon's theoretical

foundations and intended usage [8, 11, 12, 14, 50, 55]. Studies offering heuristic based recommendations for Muon, such as Muon used with momentum, or with weight decay to scale to LLMs, base them off of validation loss curve performance, or "internal metrics of instability", that are not detailed [8, 10, 11, 64]. It is unclear if these internal metrics include rigorous tests of gradient oscillations and how they explain Muon's validation loss ability to generalise as compared to Adam.

Before immediate entry of studies measuring oscillation correlated to convergence in convex settings, we propose the need to formalise investigations into the remainder cases where non-convex behaviour may lead to departure from useful mathematical prediction tools such as [26, 63] discovered challenges with, though progress continues to be made. We seek problem solving scales without restricting the scope of choices to aid predictions under convexity, that "real world" problem solving does not necessarily operate under. We therefore seek first order gradient tools that may correlate to certain loss outcomes without enforcing that favourable ideas from convex optimisation need be borrowed to form reliable predictions.

Furthermore we aim to offer practitioner value. First order optimisers are valued for their computational inexpensiveness to calculate, even if they are blind to the curvature, and use gradient estimates to address possible curvature challenges [2, 6]. Second order analyses are not typically practically used to give comprehensive insight into evolution of weight or gradient trajectories correlated to loss. Our study aims to complement however, not supersede the aforementioned studies. Future studies may probe the results of baseline experiments such as ours to conduct curvature studies as related to any results of interest at the first order level of oscillation.

We now reserve 2 studies to expand upon that are feasible in scope of analysis for the desired architecture sizes, batch dataset training sizes, and computational efficiency. They offer promising results in connection with the broader literature mentioned.

3.2.3 Gradient correlation oscillation tracking [36]

In the study of [36] gradient vector correlations across a fixed window of iterations were studied. This studies the magnitude to which two gradient vectors are opposed or agree in direction step.

3.2.3.1 The foundational formula

The basis formula proposed in the study stems from linear algebra and is required

to understand its extended applications:

$$\mu(j,k) := \frac{\langle \nabla \mathcal{L}_n(\theta_j), \nabla \mathcal{L}_n(\theta_k) \rangle}{\|\nabla \mathcal{L}_n(\theta_j)\| \cdot \|\nabla \mathcal{L}_n(\theta_k)\|} \in [-1,1], \quad \forall j,k$$

Here j,k are different positive integer pairs representing adjacent iterations. These iterations do not have to be immediate. They represent some iteration in the past compared to one in the future. Analysis of future and past gradient vectors is restricted to those within the window size chosen as $h = 5$. The numerator is the inner product (or dot product) of the past and future gradient vectors. The denominator computes the L2 norms of each vector and multiplies them together. The result is denoted as guaranteed to lie in the range of $[-1,1]$. This holds for any pair of vectors, including the same vector where $j = k$. Generally we are interested however in different vectors since the computed results of this formula of a vector on itself is the trivial boundary case of 1.

The name of the formula is also known as the *cosine similarity* measure, however like the authors we define it the *gradient correlation* measure for our discussion.

3.2.3.2 Motivational Intuition

An intuitive understanding of this formula follows from understanding how linear algebraic theory operates as useful tools in different fields. For simplicity, that holds accurate, the inner product of 2 vectors can be imagined as the perpendicular projection of any dimensional vector on the line that the second vector lies on. When we multiply the length of this projection with the length of the second vector, we obtain the dot product.

If two vectors originating from the origin, lie on the same line, the projected vector retains its full magnitude. Their dot product is simply equivalent to multiplying the L2 lengths of the vectors. If vectors are not parallel, then whenever one vector perpendicularly projects itself on another, the size always reduces. We term this projection, the component of the original vector. Therefore the multiplied lengths of the vector's lesser projection and the second vector, will be less than the pair of vector's full lengths multiplied by each other assuming they are perfectly parallel. The dot product is therefore always some fraction of the full L2 magnitudes multiplied together, unless the dot product equals the L2 magnitudes, in which case the dot product is 1. Unlike the L2 magnitudes however, the dot product is useful as it measures direction. In Euclidean space, when the projection of a vector onto the line through the origin that the second vector lies, this component may point in the opposite direction. The inner product when computed measures the negative magnitude of the projected

component and second vector multiplied together. If the original vector is not parallel, this similarly measures only a fraction of the full magnitude of both vectors when multiplied together and taking the negative signed direction.

Finally, the projected component may not exist. When the original vector pair is perpendicular, a projection of 1 vector onto the other produces a component vector of size 0, and the dot product is 0.

Therefore the denominator rescales, or more formally *normalises*, the dot product numerator. The denominator is useful as it defines the maximum value the dot product can reach in absolute magnitude. The signed direction of the dot product remains to measure agreement or disagreement in the mutual direction two vectors point to. It can also approximate disagreements in element wise gradients, if each element is perfectly opposite in signed value to its pair in the second vector. Then the normalised dot product is -1. It is important however that this is only a proxy measure. A dot product may be this value without perfect disagreement in each element.

Furthermore normalising the dot product offers an interpretation that decouples the total gradient magnitudes and focuses only on the relative strength of agreement or disagreement in magnitude and direction between two vectors. The result is that two different vector pairs of smaller magnitude, may get a normalised dot product equivalent to vectors of larger magnitude in the same positive or negative direction. The study does reintroduce discussion of the L2 gradient norms however where necessary.

We now extend how this core measure is used and consider the study's reporting scheme.

3.2.3.3 Experimental analyses from the basis

Three differing analyses are performed.

1) From iteration 5 of training, the window of size 5 can be formed. If we index from iteration 0, the gradient vector every iteration is compared against the consecutive future iteration pair. This forms $h - 1 = 5 - 1 = 4$ pairs. This measures the level of agreement in adjacent pair gradient correlation. Then a smoothing operation is performed. Each gradient correlation result is added together, and at the end divided by the total number of them in the window. The authors detail that the choice of h is therefore arbitrary controls only the level of smoothing. Within that region of time up to that iteration, the general correlational agreement of gradient vector pairs. They then plot the value of normalised gradient correlation per epoch as a dot in 2D space. A key point is

the window they use moves or more formally, *strides*, across time iterations by 1 each time. This measures the average agreement in the time region leading up to that iteration. A bigger h captures larger mean patterns in a region leading up to a point.

2) In a window *up to* a stated iteration, each gradient is correlated against every possible gradient in the window. This reveals more relationships, and possible cycles of gradient oscillation patterns beyond capturing only immediate pairs. This forms a gradient matrix, where the diagonal is all 1's when a gradient vector is paired with itself. The matrix is symmetric across the diagonal, due to the dot product taking gradient pairs of i to j is equivalent to that of taking j and matching it to i. The authors report the full matrix, however only the bottom triangle or upper triangle of the matrix requires reporting to reduce redundancy.

From their figures the window they used slides or more formally, *strides*, across iterations, without overlapping with previous windows.

Within a region of time, the extent to which gradients agree. For smoothing purposes. Flattens across a period. Can capture variation or smoothing intervals. Can measure across epochs consolidating them, or across iterations. Mention this in methodology. Adjacent pair disagreements are still taken. Smooth them.

Every gradient in a window *up to* the stated epoch is correlated against every possible gradient in the window. This reveals more relationships, broader patterns and possible cycles of gradient oscillation beyond capturing only each immediate pairs' correlation.

3) The dot product magnitude is reported without normalisation. This is to compare growth of agreements and disagreements as time progresses without bounding them between $[-1, 1]$. This is used in a layer analysis. For each weight set in different layers in a neural network, the normalised gradient correlation is measured across time. Then the pure dot product is kept, and measured simultaneously across time. This measures which layers dominate in the pure size of correlational disagreements (oscillations), or non-oscillations otherwise, that the normalised measure cannot capture. This pure dot product is still valid for analysis over the entire network, but is not introduced until comparative layer analysis.

3.2.3.4 Reported findings

The study proposes evidence of useful results, correlating to hallmark studies of edge of stability, and dynamical instability training based upon their formulation. They perform gradient descent on a full batch, an adaptation of the SGD

optimiser but not on stochastic batches to form 1 epoch. Each iteration is therefore 1 epoch.

1) The authors heuristically choose a learning rate size of 0.1 for training a simple convex linear classifier on the MNIST dataset [65]. The loss descent is a smooth curve, little to not oscillations are detectable by informal visual inspection. The normalised mean correlation μ between iterations is near 1, and maintains a flat line for 2500 epochs. In a convex setting, weights can find a step in sync using a fraction of the gradient, to converge in the consistent gradient signed direction. They do this without signs of diverging from the minimum and oscillating from the mean. Of course a few element wise weight pairs may diverge, or not match direction and strength perfectly, hence the mean correlation μ is not perfectly at 1, but still hovers extremely close.

2) Convolutional type neural networks achieve most of progress in reduction of an oscillating loss curve with oscillating gradients. Even under oscillation induced by $\eta = 0.1$, an overall minimised loss can be achieved. This occurs in their custom simple CNN, and VGG-16 on non-linear classification tasks. The datasets used are CIFAR-10, MNIST, SVHN. The same holds for residual networks. The difference they find is CNNs have a sharp transition from a beginning phase of positive correlation, to negative correlation. For RNNs this sharp transition is a beginning phase of negative correlation to positive correlation. They train ResNet-34, ResNet-18. In both cases the clear result is training loss is most minimised when oscillatory, in the corresponding negative correlation oscillation gradient scheme. The gradient matrix over the window is given over example epochs for further structural information of cyclic patterns in times of oscillation or non-oscillation.

3) For layer-wise analysis the authors report key layers, and indicate if non-reported layers have similar results or not, due to space limitations. They report the same normalised patterns across layers occur at the large learning rate scheme, for the same CNN and RNN architectures. The dot product gradient oscillations are also of highly similar order of magnitude the less deep the network is. Therefore oscillations happen relatively uniformly across layers. A caveat is that the deeper the network is, the more the oscillatory magnitude of the dot product shrinks. This is again consistent across the CNN and RNNs tested.

4) Smaller learning rate analysis such as up to 0.001 on the same tasks shows global normalised correlation patterns still hold for CNNs and RNNs as loss reduction occurs. Layer wise correlation largely matches that of global correlation but are more diverse. However the depth effects are more exaggerated. Layers increasingly close to the input layer, are orders of magnitude larger in the

dot product than the large learning rate scheme. With each depth increase approaching the final classification layer, the magnitude of shrinkage in the negative dot product correlation is also far greater. As the depth increases, for one RNN at depth 38, the gradient correlation is stable near 1, indicating the most stability is found at the final layer while the input layers contribute the most to measures of instability. The authors propose this highlights the importance of the compositional structure of deep neural networks. In their conclusion the authors further hypothesise a connection to Lipschitz constants, though do not formally connect it to earlier ideas. We briefly describe its importance in connection to related literature.

3.2.3.5 Theoretical connections to other relevant literature

The Lipschitz constant of the gradient corresponds to the steepness of the curvature measured when the gradient vector takes a step in the loss landscape [66, 67]. It is generated by the second order Hessian matrix measuring this curvature. In optimisation theory such as in convex settings it can define a threshold factor, that the learning rate or step size, must be less than in order to converge without divergence [36]. A large Lipschitz indicates an increase in sharpness of the curvature. A small Lipschitz indicates an increase in smoothness of the curvature. When a small shift in weights generates a large shift in the loss, this is the case of the large Lipschitz. When the input layer weights take the same step size shift as the output layer weights, the landscape curvature they interact differs as their relationship to a loss minimum is steeper at any point in time. The authors therefore hypothesise that the input layer parameters experience a larger Lipshitz constant, inducing a smaller feasible learning rate before divergence is experienced [36]. It takes smaller and fewer sized steps to cause oscillations. This is especially noted for the small learning rate regime. From their reported results, there are indications of this holding in the large learning rate regime, that the input layer gradients may encounter greater sharpness and accelerated divergence patterns than the output layers.

This connects to literature in formal Hessian based loss landscape analysis connected to weight gradient oscillations [30]. It further connects to the edge of stability and dynamical training stability studies mentioned prior, we recall a few in [27, 29, 34]. The authors of Muon have further studied [68] comparisons of Muon and Adam for how they can be used in conjunction with other techniques to promote desirable Lipshitz properties from the gradient throughout training while minimising loss. They study optimizers that may prevent sensitivity to weight and input perturbations. This research is ongoing, early, and not formally peer reviewed but further motivates the unified relevance of these topics.

This study overall relates to other recent emerging studies we could find such as [34], of oscillatory and smooth loss trajectory analysis to optimal or sub-optimal outcomes, in convex or non-convex settings, with small and large learning rates. They study how even under seemingly chaotic weight oscillation patterns, there are distinct phases of behaviour that correlate to loss reduction, when studied using measures that consolidate the evolution of gradients through time. These studies [34, 36] recognise the opportunity for empirical extension to stochastic regimes of training, under different optimizers beyond gradient descent and its stochastic counterpart that may attempt to suppress oscillation and chaotic behaviour such as Adam [36], and under a variation of tasks and loss functions.

A limitation acknowledged in the study is lack of analysis on the stochastic batch case. Full batch gradient descent is performed, however authors express interest of extension of their work to different optimisers, and stochastic batches.

3.2.4 Per weight oscillation oscillation tracking [31]

The study of [31], studied oscillations of both the gradients and the effective step size of weights, as formed using the gradient direction itself.

3.2.4.1 The foundational statistical method

For the basis of their methodology, oscillations are tracked per weight, and counted if there is disagreement in the update direction of each element wise weight when taken as arranged in the gradient vector. The number of a moment of non-oscillation (directional agreement) is simultaneously counted. Both these oscillatory statistics are stored for later use.

The activation of derivatives is then measured. If a weight experiences a gradient signal greater than zero, the zero activation is counted. Furthermore the non-zero activation is counted per weight.

These form the binary count measures of the study. They are coupled with visualisations that sample individual weight trajectories over train time, regardless of the layer they lie in.

3.2.4.2 Motivational intuition

The study intends to partly analyse the claim that Adam minimises step size annealing. Beyond oscillations part of the focus of the study was extending gradient saliency literature of adaptive algorithms as a long term predictor of weight behaviour.

3.2.4.3 Experimental analyses from the basis

A label classification task on the CIFAR-10 and Food101 datasets is performed. Two CNN architectures were used: MobileNet and ResNet18.

No baseline optimiser is used such as non-adaptive SGD, to compare the rates of oscillation. Furthermore it is explicitly stated that the goal was not to achieve maximal accuracy with Adam, but rather across a restricted hyperparameter set over a fixed $\eta = 0.001$, and $\beta_{1,2} \in 0.9, 0.999$, under varying batch sizes. Maximal train loss or accuracy with or without connection to validation loss may however contribute to the gradient oscillations and instability that characterises Adam's successful performance in competitive settings, that admittedly do require intensive tuning regimes. Oscillatory evidence under this regime will therefore not be acquired.

A table of average gradient oscillations of the total oscillations and average zero gradients of the total gradients is generated. This is per dataset, architecture, batch size configuration. It is unclear which fixed configuration of the Adam optimiser was used, or if this aggregates across all.

A scatter plot is used to visualise average zero gradients, and average oscillations in gradients. This is again per dataset, architecture, batch size configuration. A negative correlation is immediately reported.

Plots are used to report the trajectory analysis of a random subset sample of 100 weights from associated initial small or large partial derivative magnitude measures across iterations. They further perform complete random sampling of 100 weights. The weight value magnitude is shown across training iterations, and a heat map is used. It collapses the varying magnitude of a weight's update derivative to a strict direction choice, positive or negative.

3.2.4.4 Reported findings

Trajectory plot results are analysed through informal visual inspection, as related to gradient saliency. The study finds evidence of oscillations across the sampled weights, not predicted by their saliency measures typically used in the literature.

The study found that smaller batches have increased average *percentage* of gradient oscillations compared to larger batch sizes, which promote a greater rate of mono-directional gradients. This is likely due to the increased stochasticity of training under small batch sizes given the same epoch of training. Consideration of batch size is therefore necessary for evaluating oscillation patterns under realistic optimisation settings. This result is useful as opposed to the other studies part of this background that focus on the full batch case [26, 36]. This

indicates stochastic batches affect oscillatory regimes.

3.2.4.5 Theoretical connections to other relevant literature

The study did not theoretically connect its ideas to the edge of stability regime of training, where oscillations can still produce optimal loss behaviour [28, 29, 30, 32].

There is no connection however to the final loss or accuracy measures per activation and average oscillation regime. This would motivate whether the quality of oscillations per experiment is correlated to final performance behaviour, to ascertain if the trait is neural, desirable, or degrading to accuracy. Currently there is no indication of the relative desirability of the trait of oscillations. This of course was not the study's theoretical scope nor focus.

The study suggests that oscillation aware algorithms may be developed to speed up convergence and improve algorithm stability. To be clear the recommendation cannot be made that stable oscillations are related to increased convergence speed to a specific loss or accuracy minimum, at train, validation, or simulated test time. This suggestion may be supported by comparing behaviours captured between optimisers. This however provides opportunity for our study to evaluate this.

3.3 Inductive learning bias types

A variety of representational learning, feature learning *or* inductive biases, among others [43, 47, 69] are found to apply to neural network learning models including gradient based optimizers such as SGD. We term the converse bias, complexity bias (CB), *only* for the purposes of our study.

We characterise sections covered by our literature review by the **main** lens of study proposed, through title keywords and subsections to evaluate the strengths and weaknesses of Adam and Muon. Studies while focusing on the main idea of bias may rather isolate for an intersection of different secondary biases. This may or may not be explicitly mentioned nor realised. This leads to a requirement for careful interpretation of results and their scope of applicability. Standardization of benchmarks for bias testing is not always possible, and features as a discussion point in some studies such of [47, 69] recommending unification. This will be further discussed where required.

While other secondary biases will be briefly mentioned, the main biases we find have obvious use in the literature are as follows:

1. Distributional bias

2. Simplicity bias
3. Spurious feature bias

The classification scheme stems from the study of [43]. At the current time, these describe well the focus of Adam and Muon comparisons. An intersection over biases in one experiment may also occur with or without explicit mention. This is acknowledged to possibly occur in non-synthetic datasets with feature entanglement of attributes affecting biases [43]. We do not focus on all possible subtle biases that may implicitly exist.

3.3.1 Simplicity bias

We follow upon studies that separately show *both* Adam and Muon lack a "simplicity bias" [45, 46, 47, 69] as compared to that of SGD [38, 40, 41, 47, 69]. This measures tendency for models to learn simpler features before complex ones even when both maximally predict the target.

Simplicity bias historically existed as a more informal and subjective measure until the consolidated proposal of formal descriptions in [69], as based upon linear and non-linear boundaries. Intuitively it measures tendency for models to learn simpler features before complex ones even when both maximally predict the target [40, 69]. A feature is typically apriori labelled as complex or simple by a human practitioner, however counter-intuitive findings emerge when notions of what immediately appear simple can be challenged such as in the study of [47]. When tested the optimiser performs better in the presence of the simple feature, with poor performance when the reliance on the simple feature is revoked. Adam and Muon are found to both exhibit a lesser simplicity bias [45, 46, 47] compared to SGD [38, 40, 41, 47, 69]. These optimizers perform better in the presence of a complex feature, with poor performance when the complex feature is removed.

By [41, 69] a SB is identified as helping to explain the adversarial vulnerabilities and in out-of-distribution generalisation performance. This consequence is not remedied by adversarial training and ensemble methods on the dataset designs proposed. They find that exposure to diversity of training examples results in the neural network learning a combination of noisy but simple features, but not the variety of complex features. Therefore they propose that algorithmic innovations are required to avoid extreme SB if it stands as the reason for degraded performance (simplicity bias).

Recent studies find or emphasise that presence or lack of this trait is not to be immediately characterised as detrimental or beneficial to learning, respectively [44, 46, 47]. SB may prevent over fitting in finding simpler functions to explain data, and improve in distribution generalisation, which may transfer over to

cases where it is known OOD datasets exclude the simpler features to require the ability of specific complex explanations [41].

3.3.2 Spurious feature bias

Another strongly related study is that of spurious feature biases [40, 42]. These present some percentage up to 100% occurrence, though typically less Shah et al. [69], across examples of a feature where it is not desired that the model rely on it to make predictions, however due to the models inductive biases may do so. This preference occurs even if the non-spurious, or "core", feature is 100% correlated with the target, and the spurious feature is 100% or less correlated. This type of test is reflected in experiments where correlation strength is varied and not fixed such as of [40, 47]. It should be made clear that spurious features are labelled as such by the human practitioner. A spurious feature is decided by how the model reacts when the feature it learned to rely on is removed, to simulate reaction to real world generalisation. They may *either* be complex *or* simple in simulated datasets, as well as real world ones. In this section we will evaluate studies that motivate their choice for the spurious "undesired" feature being either complex [47] or simple [42, 44, 45, 46, 70, 71]. Practitioners therefore **explicitly** extend their own bias for what features they desire a model to learn [42, 44, 70, 72], and motivate why a spurious feature is seen as practically useful to not rely on.

3.3.3 Distributional bias

This bias indicates how well the model correctly learns a specified set of independent (features) *or* dependent (target) variables of an example as related to their distribution, or rate of occurrence, in the training dataset [43]. Different optimisers can be favoured for their ability to still learn from increasingly rare signals or more formally, *sparse* data as studies of [45, 46, 54, 73] show for Muon and Adam. It can then be tested how well the model learned representations under distribution shifts. In this case, rates of occurrence for the independent or dependent variables, different from the train dataset can be generated.

In the literature different terms can be applied, such as group imbalanced datasets, class imbalanced datasets, subgroup robustness datasets. These do directly employ learning tests on the distribution biases that can be expected from models, on train performance, and validation performance to simulate generalisation. However these studies often intersect with simplicity bias or spurious feature bias identification of models. In fixing the train time dataset distributions of such independent or dependent variables (or both), the resultant performance on validation datasets can be measured. Analysis of optimiser biases can be performed depending on if the distribution of simple or spurious

features have been shifted. This is a broad example of intersections of bias testing, whether explicit, or implicit choices, as introduced in Inductive learning bias types. How they are described, appear in practice and how choices are motivated will be further evaluated in Chronology of studies: Muon and Adam under inductive biases.

3.3.4 Other relevant biases

The prior list of biases is not necessarily exhaustive. More emerge that impact a models ability to learn for a specified set of independent and dependent variables targeted.

Positional bias has been evidenced by [43], which dictates that the order of presenting the same set of examples (including simultaneous or non-simultaneous order) to a gradient based optimiser matters in the final representation formed. This is crucial for stochastic batch optimisation techniques where order of presentation of train data batches are randomised and presented consecutively to the model.

From [43], there are findings showing that when varied these components bias learning ability on a task to different extents: architecture width, depth and related parameter count, regularisation techniques such as drop out, non-linear functions such as tanh or the rectifier, and also gradient optimiser used (AdaGrad, Adam, SGD). The inductive bias studies of Muon and Adam of course do not focus on this as the explicit subject of discussion even if implicit in the choices of these components to form baseline comparisons. The goal is designing specific real world relevant experiments where replacing one optimiser with another matters, while all the other baseline standards to solve problems that may be biased are fixed in configuration. We generally do not consider the full sweep over components or factors that bias a models learning ability, unless stated otherwise.

3.4 Chronology of studies: Muon and Adam under inductive biases

We operate in chronological order of studies of Adam and Muon that have been published, where each study references the last. We show how each attempts to isolate and study a subset of biases. We choose only the aspects of these studies that have trajectory analysis findings for us showing different loss or accuracy curves between Muon and Adam, at desired architecture scales, and datasets. Per study we evaluate the experiments performed, results reported. We then analyse the set of results and interpret their implications for our study.

3.4.1 The simplicity bias of Adam compared to SGD [46]

3.4.1.1 Experiment

3.4.1.1.1 a) Subgroup robustness benchmarks:

The datasets of these are Waterbirds [74], CelebA [75], MultiNLI [76], CivilComments [77, 78] datasets. These datasets all consist of examples of independent variables termed complex core features, and a target label variable. Within some independent variable sets there exists a spurious feature, that is not necessarily meant to 100% predict the target variable. The overall datasets are described as follows.

- Waterbirds [74] - The task is to predict bird *species* label. The core feature is the bird in the image. The spurious feature is the background of the image, land or water. This relates to spurious identification of a land or water bird species based off this feature.
- CelebA [75] - The task is to predict *blonde* or *not blonde*. The core feature is celebrity face images. The spurious feature is the gender of the celebrity, male or female.
- MultiNLI [76] - The task is for sentence pairs, to predict the relationship of a second sentence to a first: *entailment, neutral, contradiction*. The core feature is implicitly the semantics governing meaning. The spurious feature are *negation words* in pairs predicting *contradiction*.
- CivilComments [77, 78] - The task is to predict whether an online comment in the dataset is *toxic* or *non-toxic*. The core feature is implicitly the semantics governing meaning. The spurious feature are *demographic attributes* (race, religion, gender) mentioned in the comment predicting *toxicity*.

The study creates a train set of core features 100% correlated to the target, and one case where the spurious feature is only partly correlated. Crucially the exact percentage of the spurious feature is **not stated**, differing from previous experiments. A possible explanation is avoidance of complexity or workload in counting the correlational level. It is termed that the uncorrelated core and feature combinations are in "disagreement" (termed the worst group), and where they both correlate are in "agreement" (termed the best group). For validation sets a 50% balanced distribution of agreement and disagreement in features is used.

The datasets are used to form classification tasks. The ImageNet pretrained `ResNet-50` convolutional neural network [79] is used following from a previous

study [74]. The pretrained `bert-base-uncased` language model pretrained `bert-base-uncased` language model [80] is employed for the language tasks. They finetune all layers of the models.

They use larger batch sizes of 512, and 1024 then smaller sizes of 128 and 32 for the image and language datasets, respectively. The number of epochs until convergence is reported for language and image datasets.

Optimiser tuning is done over hyper parameter value options $\beta_1, \beta_2, \eta, \lambda$ where applicable for Adam or SGD. Adam has its ϵ denominator stabiliser frozen to default values. The hyper parameter sweep is not unified in number and exact value of choices across each dataset, nor well motivated. Regardless, this scheme is done more extensively for the large batch case than the small batch case, where the latter has only η, λ tuned over, with a restricted range of possibilities. The best *worst group* accuracy version of an optimiser is chosen from validation set performance. Multiple independent training runs are used for the decision, 5 and 4, for the image and language datasets respectively. Optimal hyperparameters are reported.

3.4.1.2 Results

3.4.1.2.1 a) Subgroup robustness benchmarks:

For subgroup robustness a table of results is reported, at large then small batch experiments. Each optimiser per dataset task, has its average balanced group, and worst group "test" set accuracy recorded. The best group performance is not recorded. In both batch size cases Adam comprehensively improves over SGD in worst group performance. On the average balanced group performance however SGD almost consistently performs above Adam. We can observe however that Adam is generally nearer to SGD's balanced group performance than SGD is near to Adam's worst group performance on the same dataset task.

3.4.1.3 Analysis and interpretation

It is proposed by [41, 69] that a spurious feature is *often* simple, and is often superficially correlated with the label to form a "surrogate" feature over one that is task-relevant. This motivates combined testing of performance related to these two biases that commonly occur in datasets in (a).

For (a) one reason to explain Adam's performance may of course be that Adam and SGD were tuned only over their shared hyper parameters: λ weight decay, η learning rate, and β_1 momentum. Adam has its ϵ denominator stabilizer, and β_2 frozen to default values from the original Adam paper's recommendations.

Therefore this experiments findings apply to Adam only under this partially frozen setting. The known full potential of Adam's adaptive capabilities [4, 7] in all its hyper parameters under rigorous tuning regimes for feature learning is not explored. The study remains useful however as it exposes the immediate *capability* of Adam to represent more complex features. Optimal desired behaviour was therefore reached with minimal tuning, even if the behaviour is not on overall average accuracy.

It should be noted that (a) is a study at the intersection of spurious features, simple features, and distributional biases. This aims to simulate what the authors perceive as real world conditions. Evaluations can result of trade offs when studying the expectations of performance on balanced group accuracy over worst group accuracy. These emergent expectations are based off combined feature traits, and of performance under certain distributions.

This study served to introduce the subgroup robustness benchmarks in their fullest extent of study, such that future studies reference this one, with a more limited scope, or slight extensions. We do not focus on it as it follows from the bodies of work that evaluate Adam compared to SGD. For experiments not covered, the general consensus in this study is that Adam, learns more rare features in distributions, and learns more complex features under spurious correlation tests. We now continue to evaluate a follow up study closely related to this one, with the same lead author. The study provides similar experimental designs comparing Adam and Muon.

3.4.2 Muon's Equally Learns Imbalanced Data Modalities [45]

3.4.2.1 Experiments

3.4.2.1.1 a) Subgroup robustness benchmarks:

[45] performed one experiment comparing SGD, Adam, Muon, and another optimiser Shampoo on two of the subgroup robustness benchmarks: MultiNLI, and CelebA as mentioned in the previous experiment of a) Subgroup robustness benchmarks:. The study similarly omitted exact account of the spurious feature correlation percentage.

The same specifications over the train and validation datasets are used as the previous experiment. Only the previous large batch size regime is tested for.

Hyperparameter configuration is equivalent. To reduce bias of choosing the best optimiser hyperparameter combination, the highest of this score is the best up to the max epoch budget averaged over multiple runs. The score at the final epoch

is therefore not used, but rather respects individual optimiser performance and time to convergence.

3.4.2.1.2 b) Class imbalanced datasets:

The CIFAR-10 and CIFAR-100 image datasets are used [81]. These consist of independent variables of tiny colour images in terms of pixel size dimensions (32×32), and target variables of mutually exclusive class labels. In the case of CIFAR-100, two target variables, the class (fine) label, and superclass group (broad) label exist. CIFAR-100 consists of 100 classes grouped into 20 superclasses. CIFAR-10 consists of 10 classes. There is a total of 50000 training images and 10000 test images in both datasets, spread equally across all classes [82]. For this study of [45], class imbalanced versions of the datasets are created using a ratio $R \in \{1, 5, 10, 20\}$. They sample 5 classes to be the majority classes, while the remaining are minority class. The distribution of the majority is set to R times greater than the minority class. They apply what they state is a standard data augmentation regime., of randomised flips, crops, normalisation with means and standard deviations that are dataset specific. They then split the train and test sets to follow the official CIFAR splits.

The task is a classification task. For architectures they use ResNet-18, and ResNet-50 on CIFAR-10, and CIFAR-100 respectively stating them as standard for the task stemming from the study of [83]. They fully train these models from random initialisation.

They choose a batch size of 2048 for 200 epochs. They use $\lambda = 0$ for weight decay, a cosine learning rate schedule, and tune over hyper parameter value options β_1, η where applicable for Adam or Muon, along with two other optimisers not requiring highlighted focus for our purposes. Adam has its ϵ denominator stabiliser and β_2 presumably frozen to default values, though this is **not stated** by the authors. The best *minority group* accuracy version on the maximal imbalance R setting of an optimiser is chosen from validation set performance. Multiple independent training runs are **not stated** to be used. Optimal hyperparameters, except the omitted specifications of Adam, are reported.

3.4.2.2 Results

3.4.2.2.1 a) Subgroup robustness benchmarks:

Per dataset they plot the average worst group and balanced group *validation* set accuracy across training epoch time steps for the optimal chosen optimisers. The study reported comparable average balanced group test accuracy over train time

of the optimizers, and improved worst group accuracy of *both* Adam and Muon. Muon only performs more comparably to Adam in the worst group accuracy after a greater epoch budget, and is slightly worse than Adam for MultiNLI consistently across epochs.

Average test set results as in the previous experiment, are not formally reported. It is only stated that "average test accuracy is comparable across optimizers".

3.4.2.2.2 b) Class imbalanced datasets:

For both datasets learning tasks, train loss trajectory, test minority class and majority class accuracy trajectories are separately reported for all optimisers for $R = 20$. Other settings of R are reported separately with only the average accuracy and minority class accuracy on the same plot are reported. For $R = 20$, CIFAR-10 ResNet-18 Muon achieves best minority class performance especially early in training, with similar competitively high score for the majority class. This is reflected in the differing behaviour on all trajectories. As R increases, the trajectory curve effects of Muon improving over other optimisers becomes more pronounced. Equivalent results hold for CIFAR-100, ResNet-50, except Muon has more superior performance gains starting even from $R = 1$ (no imbalance), until $R = 20$.

3.4.2.3 Analysis and interpretation

Overall, though other experiments are not covered, Muon generally performs comparably or better than Adam in the cases studied in other spurious, simplicity, and distribution bias cases tested.

One exception we term (c) is one small scale experiment where class imbalance settings are again tested beyond (b) of a language task with a transformer architecture. For majority and minority class train loss trajectory results similar to (b) for Muon occur, including its early performance increase over Adam (less epochs to convergence), while SGD is notably worse at train time. The next token prediction task however, is on a simple custom transformer architecture on the small scope TinyStories dataset. To briefly note, a key result is that Muon experienced a consistently reduced performance on the validation loss on *both* the minority and majority class, compared to *both* Adam and SGD. This performance does not match its performance on the train set. The authors do not evaluate on this comparative discrepancy across optimisers, and across optimisers on the validation loss **at all**. The proposed benefits of Muon's spectral learning abilities, if compared to other optimisers, is not demonstrated even if results appear beneficial from training results. This may offer opportunity for

exploration through oscillatory dynamics, though we briefly discuss it due to its smaller scale.

(a), (b), (c) capture varying blends of performance. Either Adam and Muon perform comparably across inductive bias varieties, or one consistently behaves different to the other in terms of trajectory analysis, whether slightly or greatly.

Again we observe (a) is at the intersection of spurious feature, simple features, and distributional biases. This is similar to the previous study of [46]. For (b) a distributional bias may be the immediate claim of study for class imbalance testing. While feature complexity nor spurious features are not studied directly, though such traits can characterise data if desired to study over. Indeed these implicit biases may influence the results such that Muon struggles to generalise for validation loss. This is only an informal hypothesis. Regardless of complexity or spurious features we check distributional learning of what we label the minority independent and dependent variable combinations. Further analysis of optimiser behaviour under distribution shifts based on this trait could be performed as done for the subgroup robustness tests (a), though not done for (b) or (c). Instead original train distributions are evaluated. This study of distributional bias in the vein of similar studies at larger scales of [73] for Adam compared to SGD, and of [54, 73] for Muon compared to Adam. Notably Muon is shown to outperform Adam in the latter studies.

There exists reason why this may be the case. A crucial distinction for the above studies is stated lack of use of Muon in the non-matrix layers such as the RMSNorm parameters, and non-dense layers such as the token embedding matrix, as recommended by Muon's authors [8, 11, 12] for theoretical reasons of as Muon is not to be used in sparse matrices due to its orthogonalising property. The study of [54] studied these recommendations further, under an extensive variety of Adam and Muon for different layer combinations of a transformer. The main finding was Muon can be applied to all feed forward, and attention head matrix matrices. Specifically, the large language model scales they study at further reveal the most benefits near full Muon use is recovered when training Muon only on the value-output (VO), and feed forward neural network (FFN) layers. Importantly the use of Adam on the embedding matrix is respected. Only unknown empirical dynamics stemming of optimiser choice other layers are studied.

Furthermore, a variant of Adam, *AdamW* with decoupled weight decay [84] is termed a crucial modification on the standard weight decay regimes previously used of Adam that resulted in the phenomena of well tuned SGD outperforming Adam in generalisation tests [84, 85]. The authors of this study [46] **do not**

state the weight decay formula variant used on Adam, and **do not** explicitly state which layers Muon does not train and if Adam trains them, and for which hyperparameter specifications. This combined hyperparameter tuning regime is not necessarily trivial, but can be more efficiently done with recommendations collected and proposed by [10, 11].

3.4.3 When Muon's Lack of Simplicity Bias Hurts its Performance [47]

3.4.3.1 Experiments

3.4.3.1.1 a) Spurious pixel with varied intensity:

The MNIST dataset [65, 86] is used. It consists of black and white images of digits, small in terms of pixel size dimensions (28×28). The independent variable is the image, and the dependent variable is the digit label from 0 to 9. There are a total of 60000 train and 10000 test images in the dataset. Within *all* independent variable sets there exists a spurious pixel feature in the image, i.e. correlation of 100% to predict the target variable. For validation sets two are formed, one fully with the spurious feature, one without. Crucially, the spurious pixel is set to varying intensities, from 1 to 6 (on the greyscale of 0 to 255, as black and white respectively), in discrete step sizes of one.

The dataset is used to form a classification task. A custom small scale CNN is used, with specifications detailed.

The batch sizes used for training are **not stated**.

SGD, Adam, Muon, all with momentum, are compared. They specify the version of Muon used as the implementation as from [8], with momentum. The study states the best performing hyperparameters are selected across multiple runs but **does not** specify them, how a hyperparameter sweep was performed, nor which hyperparameters are kept frozen or swept over. They do not state how the optimal hyperparameters are chosen such as best performance over a validation set of a particular distribution as done in [45]. They **do not** clearly state which spurious pixel intensity setting the hyperparameters were chosen for, or if reuse from the setting mainly reported on with more focus. This report will be detailed below.

3.4.3.2 Results

3.4.3.2.1 a) Spurious pixel with varied intensity:

A validation log loss, and accuracy trajectory plot is used across train time iterations, when the spurious pixel intensity (SPI) is 1 on both validation sets. A secondary plot captures the maximum accuracy on the non-spurious validation dataset, of each optimiser, at a specific SPI.

At an SPI of 1, the loss curves have distinct phases of difference, however SGD is consistently lower in loss and higher in accuracy than Muon, and Adam after some initial iterations. Muon exhibits a phenomenon where at halfway through train time, the lowest loss, and highest accuracy that matches that of Adam but not of SGD, and beyond that point Muon begins to to degrade in performance rapidly. This dipping phenomenon is highly subtle in the other optimisers, indicating the possibility of training beyond convergence where early stopping may be preferred as a strategy for all optimisers. The authors only promote this for SGD in their discussion, although Muon may potentially achieve Adam's validation performance under this modification.

When the maximum accuracy is measured across different SPI values, it is seen Muon and Adam perform comparably, and close to SGD, when their *worst* performance degradation is not inspected. Crucially, when the spurious pixel intensity is increased *all* optimisers have greatly reduced max accuracy. At an intensity of of 3.5, the authors identify a cross over point where the performance of all optimisers is the same, but reduced. The SPI value is stated as approximate and it is unclear if this exact SPI not in the discrete sequence up to 6, was tested. At an SPI of 6, the most degraded performance occurs following a continuous decline of Adam and SGD. From spurious pixel intensity threshold of 3.5 however Muon's performance stays steady and comparable.

3.4.3.3 Analysis and interpretation

We note that the authors do not impose an immediate assumption on whether a spurious pixel is simple or not, and state that it is dependent on its intensity. They observe when a spurious pixel is faint, Muon rely and Adam rely more on the feature, while SGD does not. When the SPI grows more intense, SGD relies in the feature more, while Adam not as much, and Muon the least. We find this relates to the study of [43]. Here the authors perform at the study of a distributional bias, and complexity bias. They explicitly state use of 100% spurious feature correlation, which we note is different from [45] that uses a value of 99% when they perform a somewhat similar analysis using the Colored-MNIST dataset of coloured digits, with the spurious feature being the fixed intensity colour digit and contrasting background constants per class – a scheme the authors here note may have implicitly benefited Muon. In this study of [47] use 2 validation sets to test the distributional bias, when matching the train and

test set. Furthermore varying the intensity of the pixels may impact both the distributional and complexity bias itself. In this study the authors propose that future studies vary not only the training distribution, but the feature *dominance*. The study of [43] specifies a feature prevalence bias, that has to do with the rate of equal or unequal occurrence of a feature in the dataset. An increased quantity of a feature across the distribution is one method of study, however the signal strength when a maximal and minimum scale range exists for a feature, can be tested linking to a potential extension of feature prevalence though not formally recognised. This appears to be the proposition.

Regardless, this shows simplicity is dictated by which core feature the *optimiser* finds "simpler" to learn depends on its prominence, and should not be imposed or assumed immediately by human practitioners as in the subgroup robustness experiments of [45, 46]. The authors of this study do not necessarily classify the task as a simple or complexity bias task, and simply desire to learn the core or dominant feature each optimiser is reliant on. Recall this distinction was introduced in the section: Simplicity bias.

It is crucial to note that while Muon's lack of SGDs inductive bias can hurt performance, it is unclear whether early stopping may benefit Muon. Muon can recover the maximal accuracy of SGD, even when the SPI is the lowest value, therefore its bias does not appear to harm it from that report. However SGD, when SPI is the highest value, cannot recover a maximal accuracy at any stage in training that is comparable to that of Muon. It's inductive bias appears to harm it without recovery mechanisms even as training is continued. Furthermore the spurious pixel is quite faint at maximum intensity relative to the maximum of 255. If SGD exhibits degradation based off spurious pixels, its inability to remove its dominant component bias even at subtle colour levels might indicate Muon is more robust when spurious pixels are easily dominant to SGD.

The authors hypothesise that Muon's inductive bias allows it to learn all feature components more equally than SGD. They hypothesise that SGD focuses on a single component to optimise loss over, before *then* focusing on the next component. In the case of SGD this may be what explains the resultant loss curve degradation on the validation test, as stemming from reliance on the faint spurious pixel.

We hypothesise that, to explain the loss and accuracy curves of Muon, it may be that Muon as it balances learning across certain, identified and tested components, still is influenced by the dominant digit feature that allows it to reduce loss on the validation set. As training continues Muon learns a representation that fine tunes based off all the data in the image, including any spurious pixel. This

balanced learning scheme may cause a representation to be learned in a highly complex fashion, that does not fit the validation distribution. Signs of complexity bias may apply, though the name for the inductive biases at play is not trivial to formalise, even if more objective feature dominance patterns are clear as is preferred by this study. Muon's performance degrades quickly due to more effective learning of the spurious pixel that was previously ignored, but Muon more effectively learns these components in a more balanced way given enough time, for the minority features.

Overall, the loss and accuracy curves of Adam and Muon overall show more steady growth in better outcomes over the imbalanced and preferential learning dynamics of SGD. In the trajectory of SGD we identify a period of worse loss and accuracy in initial iterations followed by rapid improvement in loss and accuracy, before the latter period of slight degradation due to possible influence of spurious elements of the dataset, not only the spurious pixel, due to over fitting.

4 Methodology

4.1 Prerequisites

We define a set of required prerequisites for our methodology. This stems from the background literature covered, but is in greater required detail and serves as a consolidated and consistent source of reference when needed. These therefore may cover global concerns over all or most experiments.

4.1.1 Oscillations tracking methodology

We fully employ the gradient correlations methodology *and* notation as detailed in the section The foundational formula.

This study allows correlating weight oscillations without direct per weight tracking or sampling. It is similar to the storage efficient, aggregated statistics of [31], while being more well correlated across training time accuracy and loss trajectory evolution as a consolidated measure. This is due to it being connected to edge of stability training phenomena on convex and non-convex tasks.

The study of [36] performed a full batch analysis. In order to extend to the stochastic batch schemes as are in realistic training schemes, we use every iteration's gradient vector across epochs. Another extended approach may be to consider smoothing of gradient correlations for each epoch, or smooth the gradient vectors to obtain the average gradient vector per epoch. Then smoothing

across windows of *epochs* may be performed. This can be done in addition to capturing the gradient correlation scalar values for each stochastic batch vector. It is unclear which scheme and level of granularity would better capture or lose information of dynamics. The only benefit of per epoch statistics being captured, would be to throw away per batch iteration statistics after each is accumulated via reduction for the averaging operation at the end. This would result in some storage efficiency, at the cost of valuable data at more granular levels.

We note that even in the most storage expensive case, the benefits over a sampling of high dimensional weights are clear. Computationally, we can consider each gradient update set for weights as existing within a corresponding matrix or vector layer. Note we consider bias vectors to be weights whether or not they are decoupled in practical architectures. For every iteration, we compute the gradient correlation between the current and previous gradient update vector. We store this result as a scalar. For 2 iterations, there is 1 gradient correlation. For 100 iterations there are 99 gradient correlations when every gradient vector is paired with the previous gradient vector, if it exists. It does not for the gradient correlation at index 0. This forms a gradient correlations history (GCH) vector of size $n-1$ for n iterations. If we consider multiple layers being tracked, then this gradient vector needs to be stored for each. Given l layers, this forms l of the size $n-1$ separate GHC vectors. We can imagine a matrix formed of $(n-1) \times l$ dimensions for row $\times$ col respectively. This is not the literal representation. However it illustrates a visualisation we can leverage for ease later.

Practically if the cost of storing this matrix of scalars is prohibitively large in dimensionality and storage is not available on a certain computational budget an amendment can be made. Instead of each scalar gradient correlation entry being added per gradient vector of a layer, we perform a reduction of the mean across the layer column axis. When expanded this directly corresponds to taking the cosine similarity measure of each layer vector pair, and averaging them out. And this further corresponds to taking the cosine similarity for longer global gradient vector pairs each composed of all the layer vectors concatenated together at that iteration. This is due to properties of linear algebra that we leverage, but do not prove.

The result is that a matrix is not stored over all iterations. We instead have a consolidated measure of global gradient vectors, that comes from the layer gradient vectors. Indeed at any point, given the GHC *matrix* over all layers and iterations, we can recover the GHC *vector* for global metrics at any time. It is therefore sufficient to store the GHC *matrix* alone. But if storage is an issue, the GHC *matrix* may be lost as the GHC global vector metric cannot recover it.

We do not aim to limit our study to only global sampling, or only sampling of specific layers, and due to the computational efficiency we propose, aim to collect data over all layers as this methodology allows.

In this way we recover the methodology of the study. The window size of h can be used over the scalar gradient correlations only for smoothing purposes (if desired) at a certain iteration. It is key to note that in our formulation the window size of h corresponds to capturing the gradient correlations over $h+1$ gradient vectors. The source study measures the window over the gradient vectors themselves for the averaging operation, only because the sequence of taking gradient correlations is not always on concurrent iterations, but over an upper triangular matrix of all possible gradient vector pairs in a window when required to show cyclical patterns. This level of representation and storage is not required however for their representation of an average of concurrent iterations up to an iteration point. Hence we make this distinction. We do not opt to use this high dimensional pairing gradient matrix. We only use this method after notable periods of oscillation are discovered in the trajectory tracking of gradient correlations up to an iteration. This requires running the training with the same seeds to reproduce the exact behaviour, and focus on capturing a window of gradient vectors instead of only the scalar values. This is similar to how the authors picked only specific notable, iteration points to show their full form with symmetry of the triangular matrix. However their choice appears heuristic, and not as a post processing step, which we may consider.

We will extend the study in one crucial fashion. If multiple runs are utilised to report the gradient correlation of a gradient vector, whether of a particular layer, or the entire network, then the mean, standard deviation, max, and min can reported from all runs instead of one as seemingly used by the study, which may not be fully representative. Therefore live tracking of gradient correlations cannot necessarily be used for our study. In other words, we do not immediately plot a layer's scalar gradient correlation at a training iteration time step. We leave all data stored, before performing layer or global metric reports. These metrics are reported as the average, std, min, max, with a box plot at an iteration time step. This replaces the simple dot used to mark the value of gradient oscillations of the study. For simplicity, only the mean may be reported to prevent a cluttered plot where desired, or under extremely stochastic, noisy settings. Or separate plots may be used for std, min, max, average instead of a unified box plot.

We take a window size of $h = 2$ and use it to take the gradient vectors of concurrent iterations and average over them to represent. At any iteration, at least 2 gradient vectors are required to be recorded in history,

In our study we propose a per layer tracking on *all* architectures studied. If not computationally feasible the global tracking may be a fallback option, or tracking of only key layers of interest such as layers playing differing roles in CNN, RNN, and transformer architectures, for example.

To reduce stochasticity of batches, the same initialisation seed for batch order will be used per optimiser.

4.1.1.1 Supporting Reporting Tools

For visualisation purposes we compile the figures or reporting scheme as based upon the study with any modifications or extensions noted, to reference as our own reporting scheme.

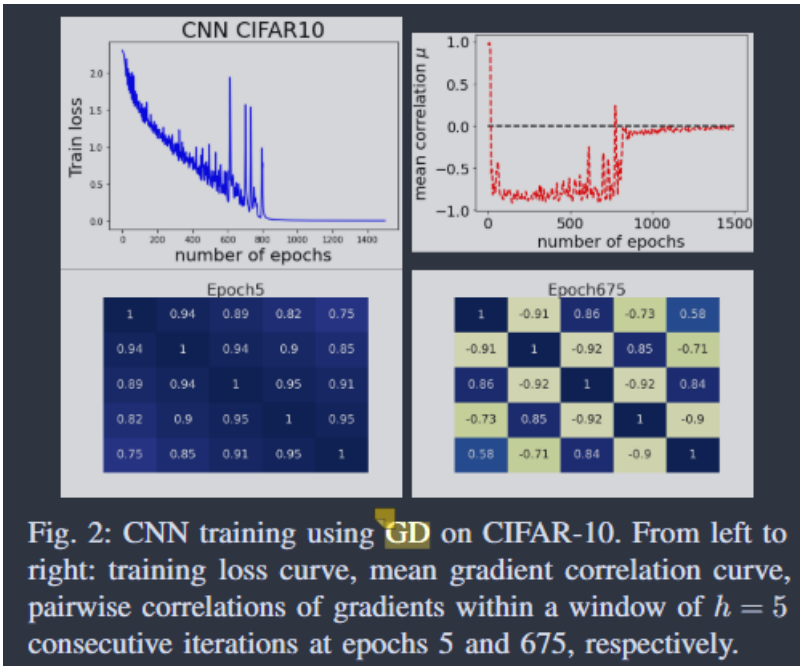


Figure 1: Global gradient correlations and loss trajectory across iterations, with window size of $h = 5$ for smoothing.

In our study we report data per iteration, then may attempt the two measures for average epoch oscillations. To obtain this trivially from the *global* GCH vector, we average gradient correlations captured over the number of iterations until 1 epoch is reached. If 1 epoch consists of 8 gradient vectors, then 7 gradient correlation pairs are present to be averaged over, not only by the chosen window size for smoothing purposes, but to capture an epoch smoothed measure (i.e. a second window size say e over the epoch). To be clear this is not the same as

value captured at that epoch iteration mark similar to all other iterations, which is still reported. This is an *additional* measure marked when an epoch is reached. We term it the epoch smoothed average (ESA). When collected per epoch, for iterations in the window of that epoch, it can be done so on a per run basis. This is stored in a separate matrix structure to the GCH, and we term it the EGCH (epoch GCH). We propose reporting this either in a separate plot, over epochs instead of the individual iterations. Alternatively we could report this in the same plot as the iterations, marking the epoch smoothed average with a special symbol.

It is crucial to now note the effects of not performing only one run as the study does. We now propose reporting only the *average* gradient correlation and dot product in the per iteration plot, across multiple runs. In other words, at the same column and row index in the GHC, we average over all runs the value at that position. Then for the EGCH, we obtain the mean measure across runs at that epoch, but also the standard deviation, min, max, etc., as with a box plot distribution. The EGCH while smoothed can now uncover more granular per epoch measures by capturing the distribution, without being as noisy as the stochastic per iteration tracking of this box plot. Smoothing applies *both* to the dot product norm and the typical gradient correlation.

To summarise for "global metrics" we report the train loss as default, then the mean run values for global correlation, then the global dot product norm, across iterations to match the train loss. We then report the mean run smoothed epoch measures. We choose if in the same plot as iterations, or on its own plot to reduce clutter. This is for one architecture and dataset combination, with hyperparameter values and train batch sizes reported.

In addition to the correlation, the dual unnormalised dot product will be stored in the GCH matrix structure proposed previously. It still stores gradient correlations history, but with the unnormalised gradients. This will be used not only for layer comparison but to capture a proxy of the global dot product norms. This can be used to compare the global norms of one optimiser compared to another, here Muon and Adam. We would like to capture the evolution of gradient norms as they agree or disagree with each other using the dot product.

All modifications for global gradient correlations reporting scheme apply. The only difference is statistics are aggregated based off the GCH matrix, not the global GCH vector. This was already covered in that explanation, that presumes trivial extension to the GCH vector. We further note a similar intention for the EGCH metrics reported as smoothed per layer.

To summarise for "layer metrics" we report the train loss as default. We use a

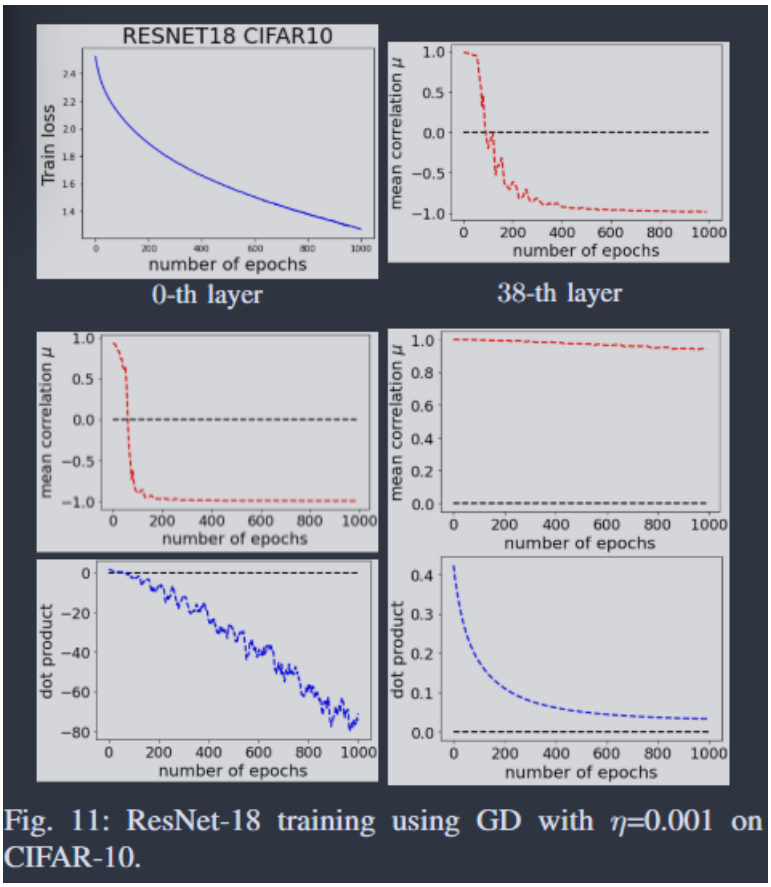


Figure 2: Overall loss trajectory, layer wise gradient correlation and dot product norm trajectory. Selected layers reported with notable dynamics. Reasons for not showing other layers is motivated where required.

figure that aligns layer metrics by relative layer index position. We report over pre-selected layers of interest and motivate what occurred on layers not shown. For each layer we report the mean run values for layer correlation, then the layer dot product norm, across iterations to match the train loss. We then report the mean run smoothed layer epoch measures. We choose if in the same plot as iterations, or on its own plot to reduce clutter. This is for one architecture and dataset combination, with hyperparameter values and train batch sizes reported.

This concludes our proposed reporting scheme. We will refer to the reporting schemes as "Global Metrics", and "Layer Metrics", and point to the associated figures. If referenced this denotes use of the entire reporting scheme as specified in the summary.

4.2 Experiments

4.2.1 Subgroup robustness setting

We use the two datasets detailed in the section a) Subgroup robustness benchmarks:, MultiNLI and CelebA. Our study will source and provide the exact spurious feature correlation percentage used.

The same specifications of how to create the train and validation datasets are used, as well as the classification task.

We use the same `ResNet-50` and `bert-base-uncased` pretrained models for image and language tasks, respectively. We fine tune all layers. We use the larger batch sizes of 512, and 1024 used, for image and language datasets, respectively. Optimiser tuning is done over the same hyperparameter value settings. A crucial difference is we specify the formula for both Muon and Adam as the following to include weight decay by simply appending this term into the update formula: $-\eta\lambda w$, for Adam, and $-\eta\lambda \mathbf{W}$ where w is a scalar valued weight input, and $\mathbf{W}$ is the weight matrix input. This is decoupled weight decay and is recommended in the standard implementation from Muon's authors [8, 9, 11], before more complex weight decay schemes are used. We perform the *same* hyperparameter configuration checks, including frozen value usage to replicate their results only with a potentially alternate, yet optimal weight decay scheme. If the authors utilised the weight decay scheme of [9], then our results should be similar. We use the same number of independent train runs to make a decision of the best *worst group* accuracy version of an optimiser over the epoch budget (i.e. the max accuracy). We report optimal hyperparameters.

Per dataset we plot the train loss and accuracy trajectories on the train dataset, an extension of the section a) Subgroup robustness benchmarks:. We do this

only when retraining from the same initialisation hyperparameters identified for optimal performance on a *new* set of train runs to capture statistics over many runs. At minimum we then report the same validation set trajectories for only one train run, if we informally decide it is representative.

We then report the "global metrics", and "layer metrics", of the correlation metrics. Recall as we desire more robust metrics, this is over multiple train runs that these are captured, even if they are matched to one loss curve. We may consider showing all loss and validation curves superimposed on each other, per optimiser, over the limited number of runs on one plot for loss. This extension is the maximal reporting case.

We recall comparable performance yet slightly differing trajectories of Adam and Muon, and seek to explain this through oscillatory measurements.

4.2.2 Class imbalanced CNN and RNN setting

As in the section [Muon's Equally Learns Imbalanced Data Modalities [@vasudeva_muon_spectral]], we use the same CIFAR datasets, train and validation set distributions, and test only for the $R = 20$, most extreme imbalance case.

The task is similarly classification. ResNet-18, and ResNet-50 on CIFAR-10 are used, not pretrained, for similar motivation to the study. For our study we particularly aim to test under a full fine tuning procedure.

We choose the same large batch size, epoch count, lack of weight decay, cosine learning rate schedule, and the same hyperparameter configurations tested, with presumed use of frozen default values for β_2. We may consider expanding the search on Adam similar to Subgroup robustness setting where different frozen values of β_2. This makes a hyperparameter sweep more expensive however and we will report if improved tuning of Adam comes from this clearly heuristic or random search choice. We will report the optimal hyperparameters, similarly chosen from best minority group accuracy, from multiple independent training runs. When choosing we take the optimal score as the max accuracy over the train epoch budget on the validation set.

For both datasets we similarly report per optimiser the same train loss trajectories, as well as what the study phrases as the held out *test* set minority and majority class trajectories. However we show these results on separate optimiser plots showing multiple optimal hyperparameter run trajectories superimposed. The test set data is never used for tuning. We may furthermore show the validation loss trajectories as in the section b) Class imbalanced datasets:. For *reporting* a choice over validation or test set will be made depending on if similar or different

dynamics are used. Again to reduce clutter, we may choose *one* representative run for these trajectories.

We then report the "global metrics", and "layer metrics", of the correlation metrics. Recall as we desire more robust metrics, this is over multiple train runs that these are captured, even if they are matched to one loss curve. We may consider showing on separated plots all loss, validation curves superimposed on each other, per optimiser, over the limited number of runs on one plot for loss. This extension is a maximal reporting case. Again we may choose *one* representative run for each trajectory curve to form the same plot that includes both optimisers as described in the section b) Class imbalanced datasets:. However this requires *starting* with the superimposed curves as the baseline and making an informed selection. This fully described protocol captures the final application decision to the previous section Subgroup robustness setting if a consolidated representation of trajectories is required. This handles both a case where an additional scheme of test trajectories, mandates simplified recommendations. For the *test* trajectory unlike the validation however which naturally uses an average over runs to decide the optimiser tuning, the test trajectory only needs to be shown once, as the representative generalisation ability. To save any space, we mainly report on the test trajectory, and state if its dynamics differ from the validation and training trajectories if placed for example in the Appendix Reporting section. This occurs only unless notable differences from train and validation dynamics occurs, where we report these.

We emphasise that consolidation over multiple runs of the correlation metrics is of course handled in their "global metrics", and "layer metrics" sections.

We recall superior Muon performance and trajectories to Adam, and seek to explain this through oscillatory measurements.

4.2.3 Spurious pixel CNN setting

We use the MNIST dataset, with the same distribution of features, as in the section a) Spurious pixel with varied intensity:. We *only* consider the minimal case of weakest Muon performance as reported in the section a) Spurious pixel with varied intensity: with SPI of 1, and optionally, at maximal case consider the case of SPI 6 for observing the switched performance of Muon and Adam as in the study of [47].

We similarly conduct a classification task. The same custom scale CNN specified is used.

As specification under this regime is missing from [47], we use the batch sizes

from the section a) Subgroup robustness benchmarks:, for the small batch size case of image datasets, 128. The number of epochs until convergence will be reported.

We copy the exact optimiser tuning and reporting regime we stated in the section Class imbalanced CNN and RNN setting. This replaces the lack of specification from [47], with choices used for the image dataset for number independent runs. Importantly, we obtain independent hyperparameter configurations for both SPI train dataset cases, as we presume the study of [47] did, though did not state the different hyperparameters.

We report the loss in log terms and the accuracy trajectory per optimiser and SPI, validation set combination as per the section a) Spurious pixel with varied intensity:, which only does it for SPI 1. We however extend the study and adopt the same protocol as in the section Class imbalanced CNN and RNN setting for handling multiple runs on the validation set.

We then report the "global metrics", and "layer metrics", of the correlation metrics.

4.2.4 Class imbalanced transformer setting

4.2.5 Appendix Reporting

4.2.5.1 Secondary measures

The main focus of our study is the main trajectory curves that coincide with the oscillatory measures. Therefore we consider hyperparameter tuning regimes, and reporting over all exact train, val, test trajectory curves secondary for reporting purposes even if used in collection. Reporting of oscillatory behaviours under optimal tuning with hyperparameters specified is the extensive focus of our study. The remaining measures are to confirm and replicate the findings of previous studies, and replace the tuning specifications if a suitable unique set is found. Hence we may place these secondary measurements in the appendix section during main reporting, to not clutter discussion except to show robust tuning protocol.

4.3 Libraries and frameworks

The language used is Python. Frameworks and libraries include: PyTorch for the custom statistical tracking measures, visualisation library for plots such as seaborn for prototyping then matplotlib for fine grained customisation of visuals, efficient matrix data storage will be done in numpy or PyTorch.

5 Planning

The official report deadline is 31 October 2026.

Intended Focus	Dates (Start – End)	Penalty Buffer Period
Dataset creation over all experiments with required distributions	16 June - 23 June	1 day
GHC baseline matrix metric storage for separate runs of cheaper subset epoch and training budget per type of architecture (CNN, RNN) small to large	23 June - 30 June	2 days
Unified trajectory curve visualisation pipeline (loss, accuracy, per train, val, test dataset)	30 June - 10 July	2 days
Visualisation of per layer iteration mean gradient correlation, dot product norms across cheap test runs	10 July - 24 July	3 days
Visualisation of global epoch smoothed norms across cheap test runs	24 July - 5 August	1 day
Hyperparameter tuning and selection over real dataset sizes	5 August - 15 August	2 days
Capture of real metrics over full dataset budgets over full pipeline	15 August - 5 September	3 days
Consolidation of results to report on as per methodology specifications	5 September - 15 September	3 days
Finish minimal research report of main results	15 September - 22 September	1 day
Appendix sections	22 September - 1 October	0 days
Final maximal research report with added appendix sections	1 October - 4 October	2 days

Dates are optimistic measures, and are penalised for any unforeseen delays. The intent is minimum proposed results finished, while remaining time is reserved for any Appendix section experiments while the main report should remain highly polished. The total penalty budget is 17 days, giving the latest finish as 21 October. Report writing is concurrent with all tasks for documentation purposes.

6 References

[1] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," DOI: 10.48550/arXiv.1412.6980

[2] X. Wen and M. Zhou, "Evolution and role of optimizers in training deep learning models," *IEEE/CAA Journal of Automatica Sinica*, vol. 11, no. 10, pp. 2039–2042, Oct. 2024. DOI: 10.1109/JAS.2024.124806 [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10664602

[3] R. Zhao, D. Morwani, D. Brandfonbrener, N. Vyas, and S. Kakade, "Deconstructing what makes a good optimizer for language models," DOI: 10.48550/arXiv.2407.07972

[4] D. Choi, C. J. Shallue, Z. Nado, J. Lee, C. J. Maddison, and G. E. Dahl, "On empirical comparisons of optimizers for deep learning,"

[5] R. M. Schmidt, F. Schneider, and P. Hennig, "International conference on machine learning," PMLR, Jul. 1, 2021, pp. 9367–9376. [Online]. Available: https://proceedings.mlr.press/v139/schmidt21a.html

[6] R. Abdulkadirov, P. Lyakhov, and N. Nagornov, "Survey of optimization algorithms in modern neural networks," *Mathematics*, vol. 11, no. 11, p. 2466, May 26, 2023. DOI: 10.3390/math11112466 [Online]. Available: https://www.mdpi.com/2227-7390/11/11/2466

[7] K. Wen, D. Hall, T. Ma, and P. Liang, "Fantastic pretraining optimizers and where to find them," DOI: 10.48550/arXiv.2509.02046

[8] *Muon: An optimizer for hidden layers in neural networks | keller jordan blog*. [Online]. Available: https://kellerjordan.github.io/posts/muon/

[9] K. Jordan, *KellerJordan/Muon*. May 4, 2026, original-date: 2024-11-09T03:29:45Z. [Online]. Available: https://github.com/KellerJordan/Muon

[10] J. Liu et al., "Muon is scalable for llm training," DOI: 10.48550/arXiv.2502.16982

[11] L. Newhouse, "Understanding muon," Citation Key: newhouse2025understanding. [Online]. Available: https://lakernewhouse.com/muon

[12] J. Bernstein, "Deriving muon," 2025, Citation Key: bernstein2025deriving. [Online]. Available: https://jeremybernste.in/writing/deriving-muon

[13] B. T. Polyak, "Some methods of speeding up the convergence of iteration methods," *USSR Computational Mathematics and Mathematical Physics*, vol. 4, no. 5, pp. 1–17, Jan. 1, 1964. DOI: 10.1016/0041-5553(64)90137-5 [Online]. Available: https://www.sciencedirect.com/science/article/pii/0041555364901375

[14] J. Bernstein and L. Newhouse, "Modular duality in deep learning," DOI: 10.48550/arXiv.2410.21265

[15] Z. Kovarik, "Some iterative methods for improving orthonormality," *SIAM Journal on Numerical Analysis*, vol. 7, no. 3, pp. 386–389, Sep. 1970. DOI: 10.1137/0707031 [Online]. Available: http://epubs.siam.org/doi/10.1137/0707031

[16] Å. Björck and C. Bowie, "An iterative algorithm for computing the best estimate of an orthogonal matrix," *SIAM Journal on Numerical Analysis*, vol. 8, no. 2, pp. 358–364, Jun. 1971. DOI: 10.1137/0708036 [Online]. Available: https://epubs.siam.org/doi/abs/10.1137/0708036

[17] A. Aich, A. B. Aich, and B. Wade, "Convergence guarantees for gradient descent in deep neural networks with non-convex loss functions," *International Journal of Computer Mathematics*, vol. 102, no. 11, pp. 1808–1823, Nov. 2, 2025. DOI: 10.1080/00207160.2025.2522349 [Online]. Available: https://www.tandfonline.com/doi/full/10.1080/00207160.2025.2522349

[18] Z. Allen-Zhu, Y. Li, and Z. Song, "A convergence theory for deep learning via over-parameterization,"

[19] Y. N. Dauphin, R. Pascanu, C. Gulcehre, K. Cho, S. Ganguli, and Y. Bengio, "Identifying and attacking the saddle point problem in high-dimensional non-convex optimization,"

[20] A. Choromanska, M. Henaff, M. Mathieu, G. B. Arous, and Y. LeCun, "The loss surfaces of multilayer networks,"

[21] S. S. Du, J. D. Lee, H. Li, L. Wang, and X. Zhai, "Gradient descent finds global minima of deep neural networks,"

[22] S. J. Reddi, S. Kale, and S. Kumar, "On the convergence of adam and beyond," 2018.

[23] N. Sato, H. Naganuma, and H. Iiduka, "Convergence bound and critical batch size of muon optimizer," DOI: 10.48550/arXiv.2507.01598

[24] R. Mulayoff, T. Michaeli, and D. Soudry, "The implicit bias of minima stability: A view from function space,"

[25] S. S. Du and X. Zhai, "Gradient descent provably optimizes over-parameterized neural networks," 2019.

[26] J. M. Cohen, A. Damian, A. Talwalkar, J. Z. Kolter, and J. D. Lee, "Understanding optimization in deep learning with central flows," DOI: 10.48550/arXiv.2410.24206

[27] J. M. Cohen, S. Kaur, Y. Li, J. Z. Kolter, and A. Talwalkar, *Gradient descent on neural networks typically occurs at the edge of stability*, Feb. 26, 2021. [Online]. Available: https://arxiv.org/abs/2103.00065v3

[28] A. Andreyev and P. Beneventano, "Edge of stochastic stability: Revisiting the edge of stability for sgd," DOI: 10.48550/arXiv.2412.20553

[29] F. Liao, A. Kolomvaki, and A. Kyrillidis, "Sgd at the edge of stability: The stochastic sharpness gap," DOI: 10.48550/arXiv.2604.21016

[30] J. M. Cohen et al., "Adaptive gradient methods at the edge of stability," DOI: 10.48550/arXiv.2207.14484

[31] H. Van Der Grijp, A. S. Bosman, and K. M. Malan, "Gecco '25 companion: Genetic and evolutionary computation conference companion," NH Malaga Hotel Malaga Spain: ACM, Jul. 14, 2025, pp. 2301–2308. DOI: 10.1145/3712255.3734344 [Online]. Available: https://dl.acm.org/doi/10.1145/3712255.3734344

[32] X. Zhu, Z. Wang, X. Wang, M. Zhou, and R. Ge, "Understanding edge-of-stability training dynamics with a minimalist example," 2023.

[33] Y. Fang and T. Kincaid, "Stability analysis of dynamical neural networks," *IEEE Transactions on Neural Networks*, vol. 7, no. 4, 1996, 996. DOI: 10.1109/72.508941 [Online]. Available: http://dx.doi.org/10.1109/72.508941

[34] K. Danovski, M. C. Soriano, and L. Lacasa, "Dynamical stability and chaos in artificial neural network trajectories along training," *Frontiers in Complex Systems*, vol. 2, May 17, 2024. DOI: 10.3389/fcpxs.2024.1367957 [Online]. Available: https://www.frontiersin.org/journals/complex-systems/articles/10.3389/fcpxs.2024.1367957/full

[35] G. Rajchakit, P. Agarwal, and S. Ramalingam, *Stability analysis of neural networks*. Singapore: Springer, 2022. DOI: 10.1007/978-981-16-6534-9 [Online]. Available: https://search.ebscohost.com/login.aspx?direct=true&scope=site&db=nlebk&db=nlabk&AN=3114314

[36] C. Morchdi, Y. Zhou, J. Ding, and B. Wang, "2023 59th annual allerton conference on communication, control, and computing (allerton)," Monticello, IL, USA: IEEE, Sep. 26, 2023, pp. 1–7. DOI: 10.1109/Allerton58177.2023.10313361 [Online]. Available: https://ieeexplore.ieee.org/document/10313361/

[37] G. Rajchakit, P. Agarwal, and S. Ramalingam, "Introduction," in G. Rajchakit, P. Agarwal, and S. Ramalingam, Eds. Singapore: Springer, 2021, pp. 1–30. DOI: 10.1007/978-981-16-6534-9_1 [Online]. Available: https://doi.org/10.1007/978-981-16-6534-9_1

[38] D. Soudry et al., "The implicit bias of gradient descent on separable data,"

[39] S. Jastrzebski et al., "International conference on machine learning," PMLR, Jul. 1, 2021, pp. 4772–4784. [Online]. Available: https://proceedings.mlr.press/v139/jastrzebski21a.html

[40] G. Qiu, D. Kuang, and S. Goel, "Complexity matters: Feature learning in the presence of spurious correlations,"

[41] B. Vasudeva, K. Shahabi, and V. Sharan, "Mitigating simplicity bias in deep learning for improved ood generalization and robustness," DOI: 10.48550/arXiv.2310.06161

[42] P. Trivedi, D. Koutra, and J. J. Thiagarajan, "A closer look at model adaptation using fea- ture distortion and simplicity bias," 2023.

[43] A. K. Lampinen, S. C. Y. Chan, and K. Hermann, "Learned feature representations are biased by complexity, learning order, position, and more," DOI: 10.48550/arXiv.2405.05847

[44] M. Pagliardini, M. Jaggi, F. Fleuret, and S. P. Karimireddy, "Agree to disagree: Diversity through disagreement for better transferability," DOI: 10.48550/arXiv.2202.04414

[45] B. Vasudeva, P. Deora, Y. Zhao, V. Sharan, and C. Thrampoulidis, "How muon's spectral design benefits generalization: A study on imbalanced data," DOI: 10.48550/arXiv.2510.22980

[46] B. Vasudeva, J. W. Lee, V. Sharan, and M. Soltanolkotabi, "The rich and the simple: On the implicit bias of adam and sgd,"

[47] S. Dragutinović and R. Ranganath, "To use or not to use muon: How simplicity bias in optimizers matters," DOI: 10.48550/arXiv.2603.00742

[48] J. Duchi, E. Hazan, and Y. Singer, "Adaptive subgradient methods for online learning and stochastic optimization,"

[49] T. Tieleman, "Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude," 2012, Issue: 2 Pages: 26 Publication Title: COURSERA: Neural Networks for Machine Learning Volume: 4. [Online]. Available: https://cir.nii.ac.jp/crid/1370017282431050757

[50] A. Orvieto and R. M. Gower, "In search of adam's secret sauce,"

[51] S. Gadat, F. Panloup, and S. Saadane, "Stochastic heavy ball," *Electronic Journal of Statistics*, vol. 12, no. 1, pp. 461–529, Jan. 2018. DOI: 10.1214/18-EJS1395 [Online]. Available: https://projecteuclid.org/journals/electronic-journal-of-statistics/volume-12/issue-1/Stochastic-heavy-ball/10.1214/18-EJS1395.full

[52] S. Ruder, "An overview of gradient descent optimization algorithms," DOI: 10.48550/arXiv.1609.04747

[53] *Ila, 6th ed. (2023)*. [Online]. Available: https://math.mit.edu/~gs/linearalgebra/ila6/indexila6.html

[54] S. Wang et al., "Muon outperforms adam in tail-end associative memory learning," DOI: 10.48550/ARXIV.2509.26030

[55] J. Bernstein and L. Newhouse, "Old optimizer, new norm: An anthology," DOI: 10.48550/arXiv.2409.20325

[56] N. Agarwal, S. Goel, and C. Zhang, "International conference on machine learning," PMLR, Jul. 1, 2021, pp. 87–99. [Online]. Available: https://proceedings.mlr.press/v139/agarwal21a.html

[57] L. Kong and M. Tao, "Stochasticity of deterministic gradient descent: Large learning rate for multiscale objective function," vol. 33, Curran Associates,

Inc., 2020, 2625–2638. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/1b9a80606d74d3da6db2f1274557e644-Abstract.html

[58] K. Nar and S. Sastry, "Step size matters in deep learning,"

[59] L. Wu and C. Ma, "How sgd selects the global minima in over-parameterized learning: A dynamical stability perspective,"

[60] M. Lu, B. Wu, X. Yang, and D. Zou, "Benign oscillation of stochastic gradient descent with large learning rates," 2024.

[61] M. Andriushchenko, F. Croce, M. Müller, M. Hein, and N. Flammarion, "A modern look at the relationship between sharpness and generalization,"

[62] Y. Pan and Y. Li, "Toward understanding why adam converges faster than sgd for transformers,"

[63] A. Damian, E. Nichani, and J. D. Lee, "Self-stabilization: The implicit bias of gradi- ent descent at the edge of stability," 2023.

[64] *Why we chose muon: Our chain of thought*, Mar. 7, 2025. [Online]. Available: https://x.com/Kimi_Moonshot/status/1897929976948965870

[65] "The mnist database of handwritten digit images for machine learning research [best of the web]," *IEEE Signal Processing Magazine*, vol. 29, no. 6, pp. 141–142, Nov. 2012. DOI: 10.1109/MSP.2012.2211477 [Online]. Available: http://ieeexplore.ieee.org/document/6296535/

[66] M.-M. Zühlke and D. Kudenko, "Adversarial robustness of neural networks from the perspective of lipschitz calculus: A survey," *ACM Computing Surveys*, vol. 57, no. 6, 142:1–142:41, Feb. 10, 2025. DOI: 10.1145/3648351 [Online]. Available: https://dl.acm.org/doi/10.1145/3648351

[67] A. Virmaux and K. Scaman, "Lipschitz regularity of deep neural networks: Analysis and efficient estimation,"

[68] L. Newhouse, R. P. Hess, F. Cesista, A. Zahorodnii, J. Bernstein, and P. Isola, "Training transformers with enforced lipschitz constants," DOI: 10.48550/ARXIV.2507.13338

[69] H. Shah, K. Tamuly, and A. Raghunathan, "The pitfalls of simplicity bias in neural networks,"

[70] P. Kirichenko, P. Izmailov, and A. G. Wilson, "Last layer re-training is sufficient for robustness to spurious correlations," 2023.

[71] N. Tsoy and N. Konstantinov, "Simplicity bias of two-layer networks beyond linearly separable data,"

[72] H. Ye, J. Zou, and L. Zhang, "International conference on artificial intelligence and statistics," PMLR, Apr. 11, 2023, pp. 8968–8990. [Online]. Available: https://proceedings.mlr.press/v206/ye23a.html

[73] F. Kunstner, A. Milligan, R. Yadav, M. Schmidt, and A. Bietti, "Heavy-tailed class imbalance and why adam outperforms gradient descent on language models,"

[74] S. Sagawa, P. W. Koh, T. B. Hashimoto, and P. Liang, "Distributionally robust neural networks for group shifts: On the importance of regularization for worst-case generalization," DOI: 10.48550/arXiv.1911.08731

[75] Z. Liu, P. Luo, X. Wang, and X. Tang, "Deep learning face attributes in the wild,"

[76] A. Williams, N. Nangia, and S. Bowman, "Proceedings of the 2018 conference of the north american chapter of the association for computational linguistics: Human language technologies, volume 1 (long papers)," New Orleans, Louisiana: Association for Computational Linguistics, 2018, pp. 1112–1122. DOI: 10.18653/v1/N18-1101 [Online]. Available: http://aclweb.org/anthology/N18-1101

[77] D. Borkan, L. Dixon, J. Sorensen, N. Thain, and L. Vasserman, "Www '19: The web conference," San Francisco USA: ACM, May 13, 2019, pp. 491–500. DOI: 10.1145/3308560.3317593 [Online]. Available: https://dl.acm.org/doi/10.1145/3308560.3317593

[78] P. W. Koh et al., "International conference on machine learning," PMLR, Jul. 1, 2021, pp. 5637–5664. [Online]. Available: https://proceedings.mlr.press/v139/koh21a.html

[79] K. He, X. Zhang, S. Ren, and J. Sun, "2016 ieee conference on computer vision and pattern recognition (cvpr)," Las Vegas, NV, USA: IEEE, Jun. 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90 [Online]. Available: http://ieeexplore.ieee.org/document/7780459/

[80] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding,"

[81] A. Krizhevsky, "Learning multiple layers of features from tiny images,"

[82] *Cifar-10 and cifar-100 datasets*. [Online]. Available: https://cave.cs.toronto.edu/kriz/cifar.html

[83] K. He, X. Zhang, S. Ren, and J. Sun, "2016 ieee conference on computer vision and pattern recognition (cvpr)," Las Vegas, NV, USA: IEEE, Jun. 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90 [Online]. Available: http://ieeexplore.ieee.org/document/7780459/

[84] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," 2019.

[85] P. Zhou, X. Xie, Z. Lin, and S. Yan, "Towards understanding convergence and generalization of adamw," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 9, pp. 6486–6493, Sep. 2024. DOI: 10.1109/TPAMI.2024.3382294 [Online]. Available: https://ieeexplore.ieee.org/document/10480574/

[86] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition,"